Trabajo Final

Alumno: Miguel A. Acosta DNI 77478992L
Curso: Ciberseguridad
Instructor: Sergio D. Ballen
Ciudad: Madrid
Fecha: 23 de marzo 2026

INFORME DE PENTESTING

INTRODUCCIÓN 3
ALCANCE Y ENTORNO DE LABORATORIO 5
METODOLOGÍA 8
ENTORNO DE LABORATORIO Y ACTIVOS EVALUADOS 14
RECONOCIMIENTO INICIAL 17
ESCANEO DE PUERTOS Y SERVICIOS 21
ENUMERACIÓN POR SERVICIO 25
IDENTIFICACIÓN DE VULNERABILIDADES 35
EXPLOTACIÓN CONTROLADA 46
MEDIDAS CORRECTIVAS APLICADAS 52
REESCANEO Y VALIDACIÓN POST-REMEDIACIÓN 57
CONCLUSIONES 63
ANEXOS TÉCNICOS 66

INTRODUCCIÓN

El presente informe de pentesting documenta la fase de evaluación técnica de seguridad realizada sobre un servidor Debian GNU/Linux 12 perteneciente al entorno de laboratorio de 4Geeks Academy, en el contexto del proyecto final de ciberseguridad orientado a la restauración, protección y validación de un sistema previamente comprometido.

Esta fase se desarrolla como complemento a la investigación forense ya efectuada, y tiene como finalidad identificar, validar y corregir una vulnerabilidad adicional distinta del vector de acceso principal evidenciado durante el incidente.

A diferencia del análisis forense, cuyo propósito fue reconstruir los hechos, identificar el servicio comprometido, preservar evidencias y determinar el alcance técnico del incidente, el pentesting se orienta a evaluar activamente la superficie de ataque residual del servidor desde una perspectiva ofensiva controlada.

En este sentido, el ejercicio no persigue la explotación destructiva del sistema, sino la validación técnica de debilidades reales presentes en la configuración y exposición de servicios, con el fin de comprender su impacto, priorizar su mitigación y verificar posteriormente la efectividad de las medidas correctivas aplicadas.

Durante las fases previas del proyecto se confirmó que el servidor analizado presentaba un estado de seguridad deficiente, con múltiples configuraciones inseguras que comprometían su robustez operativa.

Entre los hallazgos más relevantes destacaron la exposición de servicios remotos, la habilitación de acceso SSH del usuario root, la presencia de un servicio FTP gravemente inseguro y una instalación de WordPress con credenciales débiles, secretos criptográficos por defecto y permisos excesivamente permisivos sobre archivos críticos.

Asimismo, la investigación forense permitió identificar evidencia directa de acceso remoto exitoso mediante SSH con autenticación por contraseña del usuario root, lo que situó a este servicio como el vector de acceso más claramente demostrado durante el incidente.

Partiendo de ese contexto, la fase de pentesting se centra en la evaluación metódica de la infraestructura expuesta, incluyendo reconocimiento, escaneo de puertos, enumeración de servicios, análisis de configuraciones inseguras y validación controlada de una vulnerabilidad secundaria.

Para ello, se emplearon herramientas habituales de auditoría técnica en entornos de laboratorio, tales como Nmap para descubrimiento y enumeración de puertos y servicios, scripts NSE para comprobaciones seguras de exposición, así como herramientas de enumeración web como Nikto y Gobuster, destinadas a identificar recursos accesibles, configuraciones débiles y artefactos expuestos a nivel HTTP.

Estas acciones se llevaron a cabo exclusivamente en un entorno controlado, sobre una máquina autorizada para pruebas, y con fines estrictamente académicos y defensivos.

El objetivo de este informe es, por tanto, presentar de forma clara, técnica y auditable el proceso de pentesting realizado, desde la fase de reconocimiento inicial hasta la validación de la vulnerabilidad secundaria seleccionada, su impacto potencial, las medidas de remediación propuestas o aplicadas y la revalidación posterior del sistema.

De este modo, el documento no solo sirve como evidencia técnica del trabajo efectuado, sino también como apoyo a la fase final del proyecto, en la que se persigue dejar el servidor en un estado funcionalmente operativo, razonablemente endurecido y alineado con buenas prácticas de seguridad.

En consecuencia, este informe debe interpretarse como una pieza complementaria al análisis forense y al rediseño de la arquitectura de red, integrándose dentro de una estrategia más amplia de respuesta, recuperación y mejora continua. Su valor no reside únicamente en demostrar la existencia de vulnerabilidades, sino en traducir los hallazgos en acciones concretas de mitigación, priorización del riesgo y fortalecimiento del entorno tecnológico analizado.

OBJETIVO Y ALCANCE

Objetivo general

El objetivo general del ejercicio de pentesting es evaluar de manera controlada la superficie de ataque expuesta por el servidor Debian analizado, con el fin de identificar, validar y documentar una vulnerabilidad secundaria distinta del vector de acceso principal confirmado durante la fase forense.

Esta evaluación se orienta a comprender el impacto real de la debilidad detectada, justificar técnicamente su nivel de riesgo, proponer o aplicar medidas correctivas y verificar posteriormente la efectividad de dichas acciones.

En coherencia con la naturaleza del proyecto, el pentesting no se plantea como una actividad destructiva ni como una prueba de compromiso indiscriminada, sino como un ejercicio técnico de validación en un entorno de laboratorio autorizado, diseñado para reforzar la seguridad del sistema y complementar los hallazgos obtenidos durante el análisis del incidente.

Objetivos específicos

De forma específica, esta fase persigue los siguientes objetivos:

- Identificar los puertos y servicios expuestos por el servidor objetivo desde una perspectiva externa de red.
- Enumerar versiones, banners, recursos accesibles y configuraciones visibles en los servicios publicados.
- Detectar debilidades de seguridad distintas del acceso inicial confirmado en la fase forense.
- Demostrar su existencia mediante pruebas de enumeración o explotación controlada.
- Valorar su impacto potencial sobre la confidencialidad, integridad y disponibilidad del sistema.
- Aplicar o proponer medidas de mitigación concretas.
- Evaluar el sistema tras la remediación para demostrar la reducción efectiva de la superficie de ataque.

Alcance técnico

El alcance técnico del pentesting se limitó al servidor Debian identificado durante la investigación previa, accesible en la dirección “192.168.1.33”

El análisis se centró en la evaluación de los servicios expuestos en red y en la enumeración de aquellos recursos que, por configuración o diseño, pudieran representar una vulnerabilidad adicional. En particular, se contempló la revisión de:

- FTP (21/tcp)
- SSH (22/tcp)
- HTTP (80/tcp)
- Recursos asociados al entorno Apache/WordPress
- Comportamiento observable del host desde una máquina de auditoría en el mismo laboratorio

Asimismo, el proceso incluyó la correlación entre los resultados del escaneo externo y la información previamente conocida del sistema, especialmente en lo relativo a configuraciones inseguras ya evidenciadas en archivos como sshd_config, vsftpd.conf y wp-config.php.

Límites del ejercicio

Con el fin de mantener el carácter controlado, seguro y académico de la prueba, el ejercicio se realizó bajo las siguientes limitaciones:

- No se efectuaron acciones destructivas sobre la disponibilidad del sistema.
- No se realizaron ataques de denegación de servicio.
- No se ejecutaron cargas orientadas a destrucción, cifrado o alteración masiva de datos.
- No se extendió el alcance del escaneo a equipos distintos del host definido como objetivo.
- No se consideraron servicios, dominios o direcciones IP públicas ajenas al entorno del laboratorio.
- Cualquier validación de vulnerabilidad se limitó al mínimo necesario para demostrar técnicamente su existencia e impacto.

Entorno de Trabajo y Operativo

Para la ejecución del presente proyecto se asumieron las siguientes condiciones de trabajo:

- El entorno de pruebas correspondía a un laboratorio controlado y autorizado.
- El servidor objetivo era el mismo sistema previamente analizado durante la fase forense.
- La fase de análisis forense se realizó desde una estación de trabajo con Windows 11, utilizando Autopsy como herramienta principal de procesamiento y revisión de evidencias.
- La fase de reconocimiento, enumeración y validación técnica de vulnerabilidades se llevó a cabo desde una máquina Kali Linux situada en la misma red de laboratorio.
- La evidencia obtenida en la auditoría forense se empleó como contexto técnico para orientar el pentesting, aunque el presente informe se centra específicamente en la evaluación activa de la exposición residual del servidor.

Criterio de éxito

Se considerará que esta fase ha cumplido su objetivo si se logra:

- Identificar una vulnerabilidad secundaria distinta del acceso inicial documentado.
- Demostrarla mediante evidencia técnica reproducible.
- Aplicar o definir una medida correctiva proporcional al riesgo observado.
- Verificar mediante reescaneo o pruebas posteriores que dicha vulnerabilidad ha quedado mitigada o reducida a un nivel aceptable.

METODOLOGÍA

La metodología aplicada en esta fase se basó en un enfoque de pentesting técnico controlado, orientado a la identificación y validación de una vulnerabilidad secundaria distinta del vector principal de acceso confirmado durante la investigación forense.

El procedimiento adoptado combinó técnicas de reconocimiento, enumeración, análisis de exposición, validación controlada y revalidación posterior, manteniendo en todo momento el alcance limitado al entorno de laboratorio autorizado.

Desde el punto de vista operativo, el trabajo se apoyó en dos planos complementarios.

Por una parte, el análisis forense previo, realizado mediante Autopsy sobre Windows 11, permitió reconstruir la cronología del incidente, identificar configuraciones inseguras y establecer contexto técnico sobre el servidor comprometido.

Por otra, la fase de pentesting, ejecutada desde una máquina Kali Linux, se centró en observar el host objetivo desde una perspectiva externa de red, tal y como lo haría un auditor o un atacante con acceso al mismo segmento del laboratorio.

Este enfoque permitió diferenciar claramente entre:

- La evidencia histórica derivada del análisis forense.
- La exposición activa observable en el momento de la prueba.

Tipo de prueba

La evaluación se desarrolló bajo un modelo de caja gris, ya que, aunque el reconocimiento y la enumeración se realizaron como si se tratara de una auditoría externa sobre el host, se contaba previamente con información contextual relevante obtenida durante la fase forense, incluyendo:

- Dirección IP del servidor objetivo.
- Sistema operativo base.
- Servicios identificados previamente.
- Configuraciones inseguras ya observadas en archivos críticos.
- Cronología general del incidente.

Este conocimiento previo no eliminó la necesidad de validar técnicamente la exposición real del host, pero sí permitió enfocar el pentesting hacia los servicios y configuraciones con mayor probabilidad de representar una vulnerabilidad residual o secundaria.

Fases de trabajo aplicadas

La metodología seguida se estructuró en las siguientes etapas:

Reconocimiento inicial

Se verificó la disponibilidad del host objetivo y se delimitó la superficie de ataque expuesta en red. Esta fase tuvo como finalidad identificar qué puertos y servicios eran accesibles desde la máquina auditora y obtener una primera visión del sistema desde una perspectiva externa.

Escaneo y enumeración de puertos y servicios

Se realizó un reconocimiento activo del servidor mediante Nmap, con el fin de detectar puertos abiertos, servicios expuestos y versiones visibles. Posteriormente, se aplicaron scripts seguros de enumeración para ampliar la información disponible sobre FTP, SSH y HTTP.

Enumeración específica por servicio

Una vez identificados los servicios principales expuestos, se efectuó una revisión detallada por servicio:

- FTP, para validar exposición, acceso anónimo y comportamiento observable;
- SSH, para confirmar visibilidad, banner y características criptográficas;
- HTTP/Apache/WordPress, para enumerar recursos accesibles, identificar rutas sensibles y detectar debilidades de configuración.

En esta fase se emplearon herramientas como Nikto y Gobuster para complementar la información obtenida con Nmap y ampliar la visión de la superficie web expuesta.

Identificación de vulnerabilidades

A partir de los resultados obtenidos, se correlacionaron puertos, servicios, respuestas observadas y configuraciones conocidas, con el fin de identificar debilidades técnicas relevantes. Esta fase no se limitó a la mera detección de puertos abiertos, sino que buscó interpretar su impacto real en términos de riesgo.

Validación controlada de la vulnerabilidad secundaria

Una vez seleccionada la vulnerabilidad secundaria a analizar, se realizaron pruebas controladas para demostrar su existencia y su impacto potencial dentro del entorno de laboratorio. Estas acciones se limitaron al mínimo necesario para confirmar técnicamente el hallazgo, evitando cualquier operación destructiva o innecesaria.

Revalidación posterior

Tras la aplicación o definición de medidas correctivas, se contempló la repetición de escaneos y comprobaciones básicas para contrastar el estado del sistema antes y después de la remediación, con el fin de demostrar la reducción efectiva de la superficie de ataque.

Herramientas empleadas

Las principales herramientas utilizadas durante esta fase fueron las siguientes:

Nmap

Se empleó como herramienta principal de descubrimiento, escaneo de puertos, detección de servicios y enumeración segura mediante scripts NSE. Su uso permitió obtener una visión precisa de la exposición real del servidor desde la red del laboratorio.

$ nmap -sS -sV -Pn -p- 192.168.1.33
$ nmap -sV -Pn -p 21,22,80 192.168.1.33
$ nmap -sV -Pn -p 21,22,80 --script ftp-anon,ftp-syst,ssh2-enum-algos,http-title,http-enum,http-security-headers 192.168.1.33

Objetivo:

- Descubrimiento de puertos
- Identificación de servicios
- Enumeración segura
- Validación externa de exposición

Nikto

Se utilizó para el análisis de configuraciones web inseguras, detección de recursos expuestos, archivos interesantes, divulgación de versiones y ausencia de determinados controles básicos en el servicio HTTP.

$ nikto -h http://192.168.1.33

Objetivo

- Detección de recursos sensibles
- Cabeceras ausentes
- Archivos por defecto
- Configuración débil de Apache/WordPress

Gobuster

Se utilizó para enumerar directorios, ficheros y rutas web accesibles no visibles a simple vista, con el fin de identificar recursos administrativos, artefactos residuales, paneles o contenido expuesto innecesariamente.

$ gobuster dir -u http://192.168.1.33 -w /usr/share/wordlists/dirb/common.txt

Objetivo

- Identificación de rutas
- Descubrimiento de paneles, ficheros y directorios expuestos
- Ampliación de superficie de enumeración web

Utilidades de apoyo

Se emplearon además herramientas y comandos auxiliares para validación manual y correlación técnica, incluyendo pruebas básicas de conectividad, revisión de banners y contraste con la información obtenida localmente durante la fase forense.

$ ping -c 4 192.168.1.33
$ curl -I http://192.168.1.33

Objetivo

- Validación de conectividad
- Comprobación de respuesta HTTP

- Contraste manual de resultados

Principios de ejecución

La ejecución de las pruebas se rigió por los siguientes principios:

- Mínimo impacto, evitando cualquier acción destructiva sobre el sistema.
- Trazabilidad, registrando comandos, salidas y capturas de pantalla relevantes.
- Proporcionalidad, limitando las pruebas al alcance definido.
- Reproducibilidad, de modo que los hallazgos pudieran ser validados nuevamente.
- Coherencia con el análisis forense, utilizando los resultados previos como apoyo contextual, pero manteniendo independencia analítica en la validación de la exposición activa.

Criterios de interpretación

Los resultados obtenidos se interpretaron conforme a una lógica de riesgo técnico realista. En este sentido:

- Un puerto abierto no se consideró automáticamente una vulnerabilidad, sino un punto de exposición;
- Una vulnerabilidad se consideró confirmada cuando existía evidencia suficiente de:
  - Configuración insegura.
  - Acceso no restringido.
  - Enumeración sensible.
  - Validación controlada de impacto.
- La ausencia de explotación destructiva no se interpretó como ausencia de riesgo, sino como una limitación deliberada del laboratorio para preservar el entorno de trabajo.

Resultado esperado de la metodología

La metodología aplicada debía permitir:

- Identificar de manera estructurada la superficie de ataque del servidor.

- Seleccionar una vulnerabilidad secundaria técnicamente defendible.
- Demostrarla mediante evidencia reproducible.
- Relacionarla con el estado general de inseguridad del host.
- Establecer una base sólida para su posterior corrección y revalidación.

ENTORNO DE LABORATORIO Y ACTIVOS EVALUADOS

Entorno

El presente ejercicio se desarrolló en un entorno de laboratorio controlado, diseñado para simular un escenario realista de compromiso, análisis y remediación de un servidor crítico.

La infraestructura de trabajo estuvo compuesta por varios sistemas diferenciados según su función técnica, permitiendo separar adecuadamente las tareas de análisis forense, evaluación ofensiva controlada y validación operativa del host objetivo.

Desde el punto de vista metodológico, esta separación resultó esencial para mantener la coherencia del proyecto.

La evidencia histórica del incidente se procesó en un entorno específico de análisis forense, mientras que la evaluación activa de la superficie de ataque se llevó a cabo desde una máquina distinta, destinada a tareas de auditoría y pentesting.

El sistema objetivo fue el servidor Debian comprometido, que posteriormente sería sometido tanto a revisión técnica como a medidas de corrección y endurecimiento.

Plataforma de análisis forense

La fase de análisis forense se realizó desde una estación de trabajo con Windows 11, utilizando Autopsy como herramienta principal para el procesamiento de la imagen forense y la revisión de artefactos relevantes. Este entorno permitió examinar metadatos, líneas temporales, archivos críticos, logs exportados y artefactos de actividad reciente, contribuyendo a reconstruir la cronología del incidente y a identificar configuraciones inseguras, servicios sensibles y cambios ocurridos en la ventana crítica.

En este contexto, el uso de Windows 11 como plataforma de trabajo no tuvo como finalidad interactuar directamente con el servidor comprometido, sino facilitar la revisión estructurada de la evidencia mediante una herramienta forense gráfica ampliamente utilizada en entornos académicos y profesionales.

Plataforma de pentesting

La fase de evaluación ofensiva controlada se ejecutó desde una máquina Kali Linux, empleada como sistema de auditoría para el reconocimiento, el escaneo de puertos, la enumeración de servicios y la validación técnica de vulnerabilidades expuestas por el host objetivo.

Kali Linux se utilizó por su idoneidad para entornos de pentesting y por disponer de forma nativa de herramientas como Nmap, Nikto, Gobuster y otras utilidades auxiliares de reconocimiento técnico.

Esta máquina se encontraba en el mismo entorno de laboratorio que el objetivo, lo que permitió observar el servidor Debian desde una perspectiva de red externa controlada, diferenciando claramente lo que el sistema revelaba localmente de lo que realmente exponía ante otro host del mismo segmento.

Sistema objetivo

El activo principal evaluado fue un servidor con las siguientes características generales:

- Sistema operativo: Debian GNU/Linux 12 (bookworm)
- Hostname: debian
- Dirección IP observada: 192.168.1.33
- Interfaz principal: enp0s3
- Entorno: máquina virtual sobre VirtualBox

Durante la fase de análisis y validación se confirmó que este host prestaba o mantenía activos, al menos, los siguientes servicios relevantes:

- SSH
- FTP (vsftpd)
- HTTP (Apache)
- MariaDB

Asimismo, se constató la presencia de un entorno WordPress desplegado en el servidor, con múltiples debilidades de configuración ya identificadas en la fase forense y corroboradas posteriormente mediante revisión local y escaneo externo.

Red de laboratorio

El servidor objetivo formaba parte de una red privada del laboratorio basada en el rango 192.168.1.0/24, utilizando como dirección observada 192.168.1.33. La puerta de enlace y servidor DNS local identificados en la revisión de configuración correspondían a 192.168.1.1, lo que permitió reconstruir una arquitectura de red simple y plana en la fase inicial del proyecto. Sobre esa base, se diseñó posteriormente una topología mejorada y segmentada con firewall y DMZ como propuesta de corrección arquitectónica.

La máquina Kali Linux utilizada para la auditoría se encontraba dentro del mismo entorno de red, lo que hizo posible realizar escaneos, enumeraciones y validaciones controladas directamente contra la IP del servidor objetivo.

Activos evaluados durante esta fase

Aunque el foco principal del pentesting recayó sobre el servidor Debian, el análisis se centró concretamente en los siguientes activos lógicos y servicios expuestos:

Servicios de red

- 21/tcp → FTP (vsftpd)
- 22/tcp → SSH (OpenSSH)
- 80/tcp → HTTP (Apache)

Componentes de aplicación

- Entorno Apache
- Instalación WordPress
- Recursos accesibles por HTTP, incluyendo:
    - /wp-login.php
    - /wp-json
    - /readme.html
    - otros directorios o rutas enumerables

Configuraciones relevantes asociadas

- sshd_config
- vsftpd.conf
- wp-config.php
- .htaccess
- servicios habilitados al arranque
- puertos realmente expuestos desde red
- usuario con privilegios excesivos de dudosa creación

RECONOCIMIENTO INICIAL

Identificación del objetivo

La fase de reconocimiento inicial tuvo como finalidad confirmar la disponibilidad del host objetivo, identificar sus parámetros básicos de red y delimitar la superficie de ataque que sería evaluada en las fases posteriores del pentesting.

A diferencia de la auditoría forense, centrada en la evidencia histórica, este bloque se orientó a observar el servidor desde una perspectiva operativa y externa, tal como sería visible desde otro sistema del laboratorio.

El objetivo técnico de esta fase fue verificar que el servidor comprometido seguía accesible en red, confirmar su dirección IP activa y preparar la base para el escaneo detallado de puertos y servicios.

Asimismo, esta etapa permitió establecer una primera correlación entre la información conocida internamente del host y la exposición observable desde la máquina de auditoría.

Identificación del host objetivo

Como resultado de las tareas previas de análisis y validación local del sistema, se confirmó que el activo a evaluar correspondía al servidor Debian identificado como:

- Hostname: debian
- Dirección IP: 192.168.1.33
- Entorno de red: segmento privado 192.168.1.0/24

La identificación de esta dirección fue coherente con la configuración de red observada directamente en el host, donde se verificó que la interfaz principal enp0s3 operaba con la dirección 192.168.1.33/24, puerta de enlace 192.168.1.1 y configuración dinámica mediante DHCP. Esta información sirvió de punto de partida para la evaluación externa desde Kali Linux.

Validación de conectividad

Una vez identificado el host objetivo, se procedió a comprobar su accesibilidad básica desde la máquina auditora. Esta validación tuvo como finalidad descartar problemas de conectividad, confirmar que la máquina respondía dentro del mismo entorno de laboratorio y garantizar que las pruebas posteriores de reconocimiento activo se ejecutarían sobre un sistema realmente alcanzable.

Las comprobaciones iniciales de conectividad se realizaron mediante utilidades básicas de red, especialmente comandos de eco ICMP y validaciones simples de respuesta del host.

Estas pruebas no tenían por objeto descubrir vulnerabilidades por sí mismas, sino confirmar la presencia del objetivo en red y establecer una línea base de accesibilidad antes del escaneo profundo.

Comandos representativos utilizados:

$ ping -c 4 192.168.1.33

La respuesta positiva del host permitió confirmar que el sistema se encontraba operativo y accesible desde la máquina Kali Linux, habilitando así el paso a técnicas de reconocimiento activo más detalladas.

Observación preliminar de exposición

Tras validar la accesibilidad del host, la atención se centró en determinar qué servicios podían estar siendo expuestos en red. En esta fase inicial todavía no se abordó un escaneo exhaustivo, sino una aproximación preliminar destinada a preparar la enumeración posterior.

Dado que el contexto forense ya había identificado la existencia de servicios sensibles como SSH, FTP, Apache y MariaDB, el reconocimiento inicial debía confirmar si dichos servicios eran efectivamente visibles desde la red y si la exposición observada coincidía con la configuración local previamente documentada.

Este enfoque fue especialmente importante porque permitió distinguir entre:

- Servicios instalados localmente;
- Servicios activos en el sistema;
- Servicios realmente accesibles desde otro host del laboratorio.

Correlación con la fase forense

Uno de los aspectos más relevantes de esta fase fue la correlación entre el conocimiento obtenido durante la investigación forense y la observación externa del sistema.

En la etapa anterior ya se había determinado que el host mantenía activos y habilitados servicios como ssh, vsftpd, apache2 y mariadb, así como una configuración de red sencilla dentro del segmento 192.168.1.0/24.

El reconocimiento inicial del pentesting permitió usar esa información como referencia, pero sin asumir automáticamente que todos los servicios activos localmente estuvieran realmente expuestos desde red.

Esta distinción resultó metodológicamente importante, ya que el valor del pentesting no reside solo en “saber qué hay instalado”, sino en verificar qué puede ser observado, alcanzado y potencialmente explotado desde la máquina de auditoría.

Resultado del reconocimiento inicial

La fase de reconocimiento confirmó que el servidor 192.168.1.33 se encontraba accesible y operativo en la red del laboratorio. Esta validación permitió pasar a la siguiente etapa del proceso, centrada ya en la identificación sistemática de puertos abiertos, servicios expuestos y versiones detectables mediante herramientas de escaneo activo.

En consecuencia, el reconocimiento inicial cumplió su propósito principal: establecer una base de conectividad y contexto operativo suficientemente sólida para iniciar el análisis detallado de la superficie de ataque del host.

ESCANEO DE PUERTOS Y SERVICIOS

Objetivo del escaneo

Una vez confirmada la accesibilidad del servidor objetivo en la fase de reconocimiento inicial, se procedió a realizar un escaneo activo de puertos y servicios con el fin de identificar la superficie de ataque expuesta del host desde la red del laboratorio.

Esta etapa tuvo como propósito determinar qué puertos permanecían abiertos, qué servicios respondían efectivamente a conexiones externas y qué información técnica podía obtenerse mediante técnicas de enumeración seguras.

Herramienta principal empleada

La herramienta principal utilizada en esta fase fue Nmap, seleccionada por su capacidad para combinar descubrimiento de puertos, identificación de servicios, detección de versiones y enumeración técnica mediante scripts NSE.

Los comandos representativos ejecutados fueron los siguientes:

$ nmap -sS -sV -Pn -p- 192.168.1.33
$ nmap -sV -Pn -p 21,22,80 192.168.1.33
$ nmap -sV -Pn -p 21,22,80 --script ftp-anon,ftp-syst,ssh2-enum-algos,http-title,http-enum,http-security-headers 192.168.1.33

Estos comandos permitieron obtener, respectivamente:

- Una visión completa de los puertos TCP abiertos.
- Una validación más focalizada de los servicios más relevantes
- Una enumeración segura específica sobre FTP, SSH y HTTP.

Descubrimiento de puertos abiertos

El escaneo completo confirmó que el servidor presentaba, como mínimo, los siguientes puertos TCP abiertos y accesibles desde Kali Linux:

- 21/tcp → FTP
- 22/tcp → SSH

- 80/tcp → HTTP

Este resultado fue especialmente relevante porque coincidió plenamente con la revisión local previamente efectuada en el host mediante ss -tulpn y netstat -tulpn, donde ya se había constatado la presencia activa de vsftpd, sshd y apache2.

La coincidencia entre la evidencia interna y el escaneo externo refuerza la validez de los hallazgos y confirma que dichos servicios no solo estaban instalados y activos, sino también efectivamente expuestos en red.

Identificación de servicios y versiones

La detección de versiones permitió identificar con precisión los siguientes servicios:

- FTP: vsftpd 3.0.3
- SSH: OpenSSH 9.2p1 Debian 2+deb12u3
- HTTP: Apache httpd 2.4.62 ((Debian))

Desde el punto de vista técnico, este hallazgo tiene varias implicaciones. En primer lugar, confirma que el servidor mantiene visible información detallada sobre los servicios ejecutados, incluyendo su tipo, versión y en algunos casos su relación directa con Debian.

Aunque la mera exposición de un banner no constituye por sí sola una vulnerabilidad crítica, sí facilita tareas de fingerprinting y enumeración, permitiendo a un atacante perfilar el sistema con mayor precisión.

En segundo lugar, la identificación de vsftpd y OpenSSH confirmó que los dos servicios más sensibles del caso —FTP y SSH— seguían visibles desde la red, manteniendo abierta una superficie de ataque que ya había sido señalada durante la fase forense.

Enumeración técnica de FTP

El servicio más crítico identificado durante el escaneo fue FTP. La ejecución de scripts NSE orientados a este servicio permitió confirmar que:

- El puerto 21/tcp se encontraba abierto.

- El servicio respondía como vsftpd 3.0.3.
- El servidor permitía inicio de sesión anónimo.
- La conexión de control y de datos se realizaba en texto plano.

Este resultado constituye una evidencia especialmente fuerte, ya que valida externamente lo que ya se había observado internamente en vsftpd.conf.

No solo se confirmó que el servicio FTP estaba activo y expuesto, sino que además aceptaba autenticación anónima, lo que representa una debilidad crítica desde el punto de vista de confidencialidad, integridad y control de acceso.

En consecuencia, el escaneo permitió clasificar este servicio como una de las áreas de riesgo más elevadas del sistema y, al mismo tiempo, como un candidato técnicamente muy sólido para ser tratado como vulnerabilidad secundaria dentro del informe de pentesting.

Enumeración técnica de SSH

La enumeración del servicio SSH confirmó la exposición del puerto 22/tcp y permitió identificar el servidor como OpenSSH 9.2p1 Debian 2+deb12u3. Además, la enumeración de algoritmos mostró un conjunto de mecanismos criptográficos que, en términos generales, resultan compatibles con una implementación moderna de OpenSSH.

No obstante, el valor principal de este hallazgo no reside en los algoritmos negociados, sino en su correlación con la evidencia obtenida durante la fase forense y la revisión local de configuración. En el sistema ya se había identificado un sshd_config inseguro, con PermitRootLogin yes, así como evidencia en logs de autenticación de un inicio de sesión exitoso del usuario root mediante contraseña desde una dirección IP concreta.

Por tanto, aunque el escaneo de Nmap no demuestra por sí solo la explotación de SSH, sí confirma que el servicio estaba visiblemente expuesto y accesible desde red, proporcionando el contexto necesario para entender por qué constituyó un vector de acceso real durante el incidente.

Enumeración técnica de HTTP

El servicio HTTP respondió en el puerto 80/tcp como Apache httpd 2.4.62 ((Debian)). El servidor devolvió como título la página:

- Apache2 Debian Default Page: It works

Este detalle es relevante porque indica que, al menos desde la perspectiva externa del escaneo, el servidor seguía divulgando una configuración por defecto o incompletamente ajustada del servicio Apache.

A ello se suma la exposición del banner Apache/2.4.62 (Debian), lo que incrementa el nivel de fingerprinting posible sobre el sistema.

La enumeración adicional permitió identificar varios recursos asociados al entorno WordPress, entre ellos:

- /wp-login.php
- /wp-json
- /readme.html
- /wp-admin/upgrade.php

Desde un punto de vista defensivo, esto indica que el CMS era visible y parcialmente enumerable desde red, lo cual no resulta deseable en un servidor que ya había mostrado deficiencias importantes en la configuración de WordPress y del propio wp-config.php.

Aunque estos hallazgos no demuestran por sí solos una explotación web remota, sí evidencian una superficie de enumeración innecesaria y una falta de endurecimiento del servicio HTTP.

Resultado global del escaneo

La fase de escaneo de puertos y servicios permitió establecer con claridad que el servidor mantenía una exposición de red incompatible con un estado seguro. En particular, se confirmó que:

- FTP permanecía abierto, accesible y con autenticación anónima activa;
- SSH permanecía accesible desde red y seguía constituyendo un servicio crítico;
- HTTP revelaba información del servidor y permitía enumerar recursos asociados a WordPress.

Estos resultados permitieron pasar de una hipótesis basada en configuraciones locales a una validación externa real de la superficie de ataque,

sentando la base para la fase siguiente de enumeración detallada por servicio y selección de la vulnerabilidad secundaria a explotar de forma controlada.

ENUMERACIÓN POR SERVICIO

Una vez identificados los puertos abiertos y confirmada la exposición de los servicios principales del host objetivo, se procedió a realizar una enumeración técnica específica por servicio.

El objetivo de esta fase fue ampliar la información obtenida durante el escaneo inicial, validar configuraciones inseguras observables desde red y determinar qué servicios presentaban debilidades explotables o una exposición innecesaria.

La enumeración se centró en los tres servicios detectados como accesibles desde la máquina Kali Linux:

- FTP en 21/tcp
- SSH en 22/tcp
- HTTP en 80/tcp

Enumeración de FTP

Objetivo

La enumeración del servicio FTP se orientó a verificar si el servidor exponía mecanismos de autenticación inseguros, acceso anónimo, fuga de información de banner y otras debilidades observables remotamente. Esta fase resultaba especialmente relevante, ya que el análisis local previo del host ya había evidenciado una configuración crítica en vsftpd.conf, con acceso anónimo habilitado, escritura permitida y ausencia de cifrado TLS/SSL.

Comandos empleados

Enumeración técnica con Nmap:

$ sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33

Guardado de resultados en archivo:

$ sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33 -oN nmap_ftp_21.txt

Validación manual del servicio FTP:

$ ftp 192.168.1.33

Acceso de prueba con credenciales anónimas:

Name: anonymous
Password: anonymous

Comandos interactivos útiles dentro del servicio:

pwd
ls
dir
cd /
bye

Resultado observado

La enumeración confirmó que el puerto 21/tcp se encontraba abierto y que el servicio correspondía a vsftpd 3.0.3. El resultado más crítico fue la validación de:

- acceso FTP anónimo permitido
- sesión establecida satisfactoriamente con el usuario ftp
- transmisión de datos y canal de control en texto plano

El resultado observado en Nmap fue equivalente a:

```
21/tcp open  ftp    vsftpd 3.0.3
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   Control connection is plain text
|   Data connections will be plain text
|_  vsFTPd 3.0.3 - secure, fast, stable
```

Interpretación técnica

Este hallazgo constituye una vulnerabilidad crítica. La posibilidad de acceso anónimo al servicio FTP implica que un usuario no autenticado puede interactuar con el servidor a través de un protocolo históricamente inseguro. A ello se suma el

hecho de que la conexión opera sin cifrado, exponiendo credenciales y contenido en tránsito.

La gravedad del hallazgo aumenta al correlacionarlo con la revisión local de vsftpd.conf, donde ya se había confirmado:

- anonymous_enable=YES
- local_enable=YES
- write_enable=YES
- ssl_enable=NO

Por tanto, la enumeración externa no solo confirmó la existencia del servicio, sino que validó operativamente una de las configuraciones más débiles del sistema.

Valor para el pentest

Desde el punto de vista del informe, FTP constituye un candidato excelente a vulnerabilidad secundaria, ya que:

- es distinto del acceso inicial forense confirmado por SSH;
- puede demostrarse externamente con evidencia técnica reproducible;
- su impacto es fácil de justificar;
- y su remediación posterior es clara y verificable.

Enumeración de SSH

Objetivo

La enumeración del servicio SSH tuvo como finalidad confirmar su exposición en red, identificar la versión del servicio y analizar las características criptográficas visibles desde el exterior. Aunque la fase forense ya había evidenciado que SSH fue el servicio comprometido durante el incidente, en esta etapa el interés se centró en documentar su estado como servicio expuesto dentro de la superficie de ataque remanente del servidor.

Comandos empleados

Detección de servicio y versión:

$ sudo nmap -sV -Pn -p 22 192.168.1.33

Enumeración de algoritmos SSH:

$ sudo nmap -sV -Pn -p 22 --script ssh2-enum-algos 192.168.1.33

Guardado de resultados en archivo:

$ sudo nmap -sV -Pn -p 22 --script ssh2-enum-algos 192.168.1.33 -oN nmap_ssh_22.txt

Validación manual de conectividad:

$ ssh debian@192.168.1.33

Prueba manual orientada a usuario privilegiado en laboratorio controlado:

$ ssh root@192.168.1.33

Resultado observado

La enumeración confirmó que el puerto 22/tcp se encontraba abierto y que el servicio respondía como:

- OpenSSH 9.2p1 Debian 2+deb12u3

Asimismo, la enumeración de algoritmos reveló un conjunto moderno de mecanismos criptográficos, incluyendo:

- curve25519-sha256
- sntrup761x25519-sha512@openssh.com
- aes128-ctr
- aes256-ctr
- aes128-gcm@openssh.com
- aes256-gcm@openssh.com
- ssh-ed25519
- rsa-sha2-256

- rsa-sha2-512

Interpretación técnica

Desde una perspectiva puramente criptográfica, la exposición del servicio no mostró de inmediato algoritmos obsoletos extremos ni banners anómalos. Sin embargo, la relevancia real de este hallazgo no se encuentra tanto en el detalle de algoritmos, sino en su correlación con la evidencia forense y con la configuración local del host.

En fases previas ya se había demostrado que:

- el servicio SSH estaba activo y habilitado;
- sshd_config permitía el acceso del usuario root;
- existió un evento real de acceso remoto exitoso:
  - Accepted password for root from 192.168.0.134

En consecuencia, la enumeración de SSH confirma que el servicio continuaba siendo visible, accesible y técnicamente explotable desde red, incluso aunque la vulnerabilidad principal asociada a él ya hubiera sido documentada en el análisis forense.

Valor para el pentest

En esta fase, SSH se trata más como un servicio expuesto de alto riesgo ya conocido que como la vulnerabilidad secundaria principal del pentest. Su inclusión en la enumeración es obligatoria para reflejar adecuadamente la superficie de ataque, pero la vulnerabilidad secundaria seleccionada para validación controlada se orienta preferentemente a otro servicio, a fin de no duplicar el núcleo de la narrativa forense.

Enumeración de HTTP / Apache / WordPress

Objetivo

La enumeración del servicio HTTP tuvo como finalidad identificar el software servidor expuesto, los recursos accesibles públicamente, posibles directorios o ficheros sensibles, información de banners y elementos del entorno WordPress visibles desde red.

Esta fase fue especialmente importante porque el análisis local ya había evidenciado múltiples debilidades en wp-config.php, permisos inseguros en el entorno web y una instalación del CMS con credenciales y secretos criptográficos deficientes.

Comandos empleados

Enumeración básica con Nmap:

$ sudo nmap -sV -Pn -p 80 --script http-title,http-enum,http-security-headers 192.168.1.33

Guardado de resultados en archivo:

$ sudo nmap -sV -Pn -p 80 --script http-title,http-enum,http-security-headers 192.168.1.33 -oN nmap_http_80.txt

Comprobación manual de cabeceras HTTP:

$ curl -I http://192.168.1.33

Recuperación manual del contenido principal:

$ curl http://192.168.1.33

Validación de recursos concretos:

$ curl -I http://192.168.1.33/wp-login.php
$ curl -I http://192.168.1.33/wp-json
$ curl -I http://192.168.1.33/readme.html
$ curl -I http://192.168.1.33/robots.txt

Enumeración web con Nikto

Comando empleado:

$ nikto -h http://192.168.1.33

Guardado de resultados:

$ nikto -h http://192.168.1.33 -o nikto_http_192.168.1.33.txt -Format txt

Finalidad técnica:

- Detectar archivos y rutas interesantes;
- Identificar banners y divulgación de versión;
- Revisar configuraciones HTTP inseguras;
- Detectar recursos por defecto o innecesariamente publicados.

La enumeración web mediante Nikto confirmó que el servidor HTTP exponía un entorno WordPress claramente identificable y presentaba varias deficiencias de hardening. En la fase inicial se observaron, entre otros hallazgos, la divulgación del banner Apache/2.4.62 (Debian), la exposición de recursos como wp-login.php, license.txt, readme.html, wp-links-opml.php y robots.txt, así como la ausencia de varias cabeceras de seguridad recomendables.

También se identificó navegación del directorio wp-content/uploads/, lo que ampliaba la superficie de enumeración y el riesgo de fuga de información.

Enumeración de contenido con Gobuster

Comando empleado:

$ gobuster dir -u http://192.168.1.33 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,bak

Guardado de resultados:

$ gobuster dir -u http://192.168.1.33 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,bak -o gobuster_http_192.168.1.33.txt

Finalidad técnica:

- descubrir directorios y ficheros accesibles;
- detectar paneles, backups, archivos de prueba o recursos residuales;
- ampliar la superficie de enumeración más allá de la navegación manual.

La enumeración de contenido mediante Gobuster reforzó los hallazgos obtenidos con Nmap y Nikto, permitiendo identificar múltiples recursos accesibles del entorno WordPress en la fase inicial del análisis. Entre las rutas

más relevantes detectadas se encontraban `wp-admin`, `wp-content`, `wp-includes`, `wp-login.php`, `xmlrpc.php`, `readme.html`, `license.txt`, `robots.txt` y `sitemap.xml`. Asimismo, se observó que el directorio `wp-content/uploads` era navegable, lo que incrementaba el riesgo de exposición de contenido y ampliaba la superficie de enumeración disponible para un atacante.

También se detectaron redirecciones hacia `localhost`, lo que apuntaba a una posible mala configuración de la URL base del CMS o del entorno web.

Resultado observado con Nmap

El servicio HTTP respondió en el puerto 80/tcp como:

- Apache httpd 2.4.62 ((Debian))

El título devuelto fue:

- Apache2 Debian Default Page: It works

La enumeración adicional detectó la presencia de varios recursos asociados al entorno WordPress, entre ellos:

- /wp-login.php
- /wp-json
- /readme.html
- /wp-admin/upgrade.php

El banner HTTP visible fue:

- Apache/2.4.62 (Debian)

Interpretación técnica

La respuesta observada en el servicio web revela varios problemas de endurecimiento:

Divulgación de información del servidor

El banner y el título de la página principal permiten identificar fácilmente:

- tipo de servidor web;
- versión concreta de Apache;
- relación con Debian.

Aunque esto no constituye una vulnerabilidad crítica por sí solo, sí facilita tareas de fingerprinting y enumeración.

Exposición innecesaria de recursos WordPress

La visibilidad desde red de recursos como:

- /wp-login.php
- /wp-json
- /readme.html

Implica que el CMS es reconocible y enumerable desde el exterior. En particular, readme.html no aporta valor operativo al servicio y sí puede ayudar a identificar plataforma, estructura o despliegue.

Configuración web no endurecida

La coexistencia entre la página por defecto de Apache y la visibilidad de rutas WordPress sugiere una configuración poco afinada del entorno HTTP, al menos desde la perspectiva del escaneo externo. Esto puede reflejar:

- Despliegue parcial
- Configuración por defecto no eliminada
- Falta de saneamiento del entorno web
- Endurecimiento incompleto del servicio Apache.

Relación con la revisión local

Estos hallazgos web deben interpretarse junto con la evidencia local ya conocida del servidor, en la que se confirmó:

- wp-config.php con permisos 777
- Contraseña débil de base de datos (123456)
- salts por defecto
- Configuración WordPress insegura.

Por tanto, aunque la enumeración HTTP no demuestra por sí sola explotación remota del CMS, sí confirma que el entorno WordPress se encontraba visible

desde red y presentaba una superficie de exposición innecesaria y mal endurecida.

IDENTIFICACIÓN Y CLASIFICACIÓN DE VULNERABILIDADES

Criterio de identificación

La identificación de vulnerabilidades se realizó a partir de la correlación entre tres fuentes principales de evidencia:

- El análisis forense previo, efectuado sobre la imagen del sistema mediante Autopsy.
- La revisión local del host Debian, incluyendo configuraciones, servicios, permisos, logs y artefactos del sistema.
- La enumeración externa realizada desde Kali Linux, mediante herramientas como Nmap, Nikto y Gobuster.

Bajo este enfoque, una vulnerabilidad no se consideró confirmada únicamente por la presencia de un puerto abierto, sino por la concurrencia de uno o varios de los siguientes factores:

- Exposición real del servicio en red.
- Configuración insegura del servicio o del componente asociado.
- Evidencia directa de acceso exitoso o abuso.
- Impacto técnico razonable sobre confidencialidad, integridad o disponibilidad.
- Posibilidad de validación controlada dentro del laboratorio.

A partir de este criterio, las debilidades identificadas se organizaron en dos grupos:

1. Vulnerabilidad explotada, es decir, aquella que cuenta con evidencia directa de haber sido utilizada durante el incidente.
2. Vulnerabilidad secundaria, validada durante la fase de pentesting como debilidad distinta del vector principal de acceso.
3. Hallazgos adicionales, relevantes para el endurecimiento y la reducción de la superficie de ataque.

Vulnerabilidad explotada: acceso remoto inseguro por SSH

Descripción

La vulnerabilidad explotada identificada con mayor nivel de certeza fue la exposición insegura del servicio SSH, configurado de manera que permitía el acceso remoto del usuario root.

La revisión local del archivo sshd_config confirmó que el sistema mantenía habilitado el inicio de sesión privilegiado, mientras que el análisis de los registros del servicio sshd evidenció un acceso remoto exitoso mediante contraseña desde una dirección IP externa al host.

Este hallazgo constituye el vector de acceso principal más claramente demostrado durante el incidente y representa la vulnerabilidad ya materializada en la fase forense.

Evidencia técnica

Exposición del servicio

El escaneo externo con Nmap confirmó que el servicio SSH se encontraba accesible en red a través del puerto:

- 22/tcp → OpenSSH 9.2p1 Debian 2+deb12u3

Comando representativo:

$ sudo nmap -sV -Pn -p 22 --script ssh2-enum-algos 192.168.1.33

Configuración insegura del servicio

La revisión del archivo sshd_config mostró la directiva:

- PermitRootLogin yes

Comando representativo:

$ sudo grep -E "PermitRootLogin|PasswordAuthentication|AllowUsers|MaxAuthTries" /etc/ssh/sshd_config

Evidencia directa de explotación

La consulta de los registros del servicio SSH mediante journalctl mostró el siguiente evento:

- Accepted password for root from 192.168.0.134
- Seguido de: session opened for user root(uid=0)

Comando representativo:

$ sudo journalctl -u ssh --since "2024-10-08 17:40:00" --until "2024-10-08 17:42:00" --no-pager

Interpretación técnica

La combinación de exposición del servicio, configuración insegura y evidencia directa en logs permite concluir que el atacante accedió al sistema a través de SSH, utilizando autenticación por contraseña sobre la cuenta root. Este hallazgo reviste una importancia central, ya que no se trata de una debilidad potencial, sino de una vulnerabilidad efectivamente explotada.

No se localizaron, en la revisión realizada, evidencias concluyentes de fuerza bruta o múltiples intentos fallidos previos; sin embargo, esta ausencia no invalida el hecho principal: el acceso remoto exitoso a una sesión root a través de un servicio expuesto e insuficientemente endurecido.

Impacto

El impacto de esta vulnerabilidad es crítico, ya que otorga acceso administrativo completo al sistema comprometido. Desde ese nivel de privilegio, un atacante puede:

- Modificar archivos del sistema;
- Alterar configuraciones de servicios;
- Crear o eliminar cuentas;
- Instalar herramientas adicionales;
- Mantener persistencia;
- Manipular el entorno web y los servicios publicados.

Clasificación

Crítica

Vulnerabilidad secundaria seleccionada: FTP anónimo expuesto y sin cifrado

Descripción

Como vulnerabilidad secundaria del pentesting se seleccionó la exposición insegura del servicio FTP, identificado como vsftpd 3.0.3, con acceso anónimo permitido y tráfico en texto plano.

Esta debilidad fue elegida por ser distinta del vector principal de acceso documentado en la fase forense y por contar con una validación externa clara y reproducible.

La gravedad del hallazgo se incrementa al correlacionarlo con la configuración local de vsftpd.conf, donde se observó:

- anonymous_enable=YES
- local_enable=YES
- write_enable=YES
- ssl_enable=NO

Evidencia técnica

Exposición del servicio

El escaneo externo con Nmap confirmó que el puerto 21/tcp estaba abierto y que el servicio correspondía a vsftpd 3.0.3.

Comando representativo:

$ sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33

Acceso anónimo confirmado

La enumeración segura con NSE devolvió el resultado:

- Anonymous FTP login allowed (FTP code 230)

Además, el script ftp-syst mostró que:

- La conexión de control era en texto plano;
- Las conexiones de datos eran en texto plano.

Correlación con configuración local

La revisión de vsftpd.conf confirmó que el servicio permitía acceso anónimo, escritura y ausencia de cifrado.

Comando representativo:

$ sudo grep -E "anonymous_enable|local_enable|write_enable|ssl_enable" /etc/vsftpd.conf

Interpretación técnica

El servicio FTP constituye una vulnerabilidad crítica, ya que permite acceso no autenticado a través de un protocolo inseguro y sin cifrado. Incluso sin necesidad de confirmar en esta fase una modificación efectiva de archivos por parte de un atacante, la simple validación de acceso anónimo ya representa una exposición severa e injustificada.

A nivel de diseño defensivo, esta debilidad resulta especialmente grave porque:

- Amplía innecesariamente la superficie de ataque.
- Carece de cifrado.
- Permite enumeración y potencial acceso a contenido.
- Por configuración local, estaba preparado para admitir escritura.

Impacto

El impacto potencial incluye:

- Acceso indebido a recursos publicados por el servicio FTP;
- Modificación, subida o sustitución de archivos si la escritura está operativamente habilitada;
- Interceptación de tráfico por operar en texto plano;
- Utilización del servicio como punto de entrada alternativo o de transferencia no autorizada.

Clasificación

Crítica

Vulnerabilidad adicional: configuración insegura de WordPress

Descripción

La revisión local del entorno WordPress confirmó una configuración gravemente insegura del archivo wp-config.php, incluyendo:

- Credenciales débiles de base de datos;
- Salts y claves criptográficas por defecto;
- Permisos excesivamente permisivos sobre el archivo crítico.

En concreto, se observó:

- DB_PASSWORD = '123456'
- Salts con el valor put your unique phrase here
- Permisos 777 sobre wp-config.php

Evidencia técnica

Comandos representativos:

$ sudo ls -l /var/www/html/wp-config.php
$ sudo cat /var/www/html/wp-config.php

Interpretación técnica

Aunque esta debilidad no se ha establecido como el vector principal del incidente, sí constituye un hallazgo de alto riesgo dentro del entorno web. La combinación de credenciales triviales, secretos por defecto y permisos de escritura global compromete de forma severa la integridad y confidencialidad del CMS y de la base de datos asociada.

Impacto

- Exposición o modificación de credenciales del CMS;
- Debilitamiento de las sesiones y del modelo de autenticación de WordPress;
- incremento del riesgo de manipulación del entorno web si el sistema vuelve a ser accedido.

Clasificación

Alta

Vulnerabilidad adicional: exposición de HTTP y enumeración innecesaria de WordPress

Descripción

La enumeración del servicio HTTP reveló que el servidor web exponía:

- Banner Apache/2.4.62 (Debian)
- Título Apache2 Debian Default Page: It works
- Recursos WordPress accesibles desde red:
  - /wp-login.php
  - /wp-json
  - /readme.html
  - /wp-admin/upgrade.php

Evidencia técnica

Comando representativo:

$ sudo nmap -sV -Pn -p 80 --script http-title,http-enum,http-security-headers 192.168.1.33

Interpretación técnica

Este hallazgo no demuestra por sí solo explotación, pero sí evidencia una falta de hardening web y una superficie de enumeración innecesaria. La visibilidad de la página por defecto de Apache, junto con recursos propios de WordPress, facilita la identificación del stack tecnológico y la localización de puntos de entrada administrativos.

Impacto

- Divulgación de información técnica del servidor;
- Reconocimiento facilitado del entorno WordPress;
- Exposición innecesaria de recursos administrativos o informativos.

Clasificación

Media

Vulnerabilidad adicional: permisos inseguros en archivos .htaccess

Descripción

La revisión del webroot mostró que tanto el archivo .htaccess principal como el correspondiente al plugin Akismet presentaban permisos 777.

Evidencia técnica

Comando representativo:

$ sudo ls -l /var/www/html/.htaccess /var/www/html/wp-content/plugins/akismet/.htaccess

Interpretación técnica

El contenido de ambos archivos no mostró reglas maliciosas evidentes; sin embargo, los permisos 777 suponen una debilidad relevante de integridad, ya que permitirían la modificación arbitraria de reglas de reescritura, redirecciones o comportamiento del servidor web por parte de cualquier usuario o proceso con acceso al sistema.

Impacto

- Modificación no autorizada de reglas web;
- Introducción potencial de redirecciones maliciosas;
- Alteración del comportamiento del CMS o del servidor Apache.

Clasificación

Media

Vulnerabilidad adicional: exceso de servicios habilitados y activos

Descripción

La revisión del sistema evidenció un conjunto excesivo de servicios activos o habilitados para un host con rol de servidor, incluyendo componentes de escritorio y servicios no esenciales, entre ellos:

- vsftpd
- avahi-daemon
- bluetooth
- cups
- lightdm
- ModemManager
- wpa_supplicant

Evidencia técnica

Comandos representativos:

$ systemctl list-unit-files --type=service
$ systemctl --type=service --state=running

Interpretación técnica

Este hallazgo no implica por sí mismo explotación, pero sí refleja una postura de seguridad deficiente y una superficie de ataque innecesariamente ampliada. La presencia de entorno gráfico y servicios de escritorio en un servidor web/administrativo no endurecido complica el análisis, aumenta el ruido operativo y eleva el riesgo general.

Impacto

- Mayor superficie de ataque;
- Más componentes susceptibles de error o abuso;

- Complejidad innecesaria en un sistema que debería estar minimizado.

Clasificación

Media

Hallazgos no confirmados como malware o rootkit

Durante la revisión del sistema también se investigaron indicios de:

- rootkits;
- ejecutables sospechosos en temporales;
- persistencia por cron;
- persistencia por authorized_keys;
- archivos ocultos o web shells en el webroot;
- binarios SUID/SGID anómalos;
- uso de /etc/ld.so.preload.

Con la evidencia disponible, no se identificó un rootkit o malware residente inequívoco. Tampoco se localizaron authorized_keys en /root o en perfiles de usuario, ni scripts claramente maliciosos en /tmp, /var/tmp o /dev/shm. Los binarios SUID/SGID y las capabilities revisadas fueron compatibles con un sistema Debian estándar.

Por tanto, el escenario observado se corresponde con mayor probabilidad con una intrusión facilitada por configuraciones inseguras, más que con la implantación demostrada de un malware sofisticado o persistente.

Clasificación consolidada de vulnerabilidades

ID	Vulnerabilidad	Servicio/Componente	Severidad	Estado
V-00	Acceso remoto inseguro por SSH con login root y evidencia de explotación	OpenSSH / 22-tcp	Crítica	Explotada
V-01	Acceso anónimo a FTP y tráfico sin cifrado	vsftpd / 21-tcp	Crítica	Confirmada
V-02	wp-config.php con credenciales débiles, salts por defecto y permisos 777	WordPress	Alta	Confirmada
V-03	Exposición de información y recursos WordPress desde HTTP	Apache / HTTP / WordPress	Media	Confirmada
V-04	Permisos inseguros en archivos .htaccess	Entorno web	Media	Confirmada
V-05	Exceso de servicios habilitados y activos	Sistema operativo	Media	Confirmada

.

VALIDACIÓN Y EXPLOTACIÓN CONTROLADA DE LA VULNERABILIDAD

Objetivo de la fase

Una vez identificada y clasificada la vulnerabilidad secundaria seleccionada, se procedió a su validación técnica en un entorno de laboratorio controlado.

El objetivo de esta fase no fue comprometer destructivamente el sistema, sino demostrar de forma reproducible que la debilidad detectada era real, accesible y explotable desde la red, así como estimar su impacto potencial sobre la seguridad del servidor.

En coherencia con el alcance definido para este proyecto, la explotación se limitó al mínimo necesario para confirmar técnicamente el hallazgo. No se realizaron acciones orientadas a degradar la disponibilidad del sistema, alterar masivamente su contenido ni implantar persistencia adicional.

La validación se centró exclusivamente en demostrar la existencia operativa del problema y su riesgo asociado.

Vulnerabilidad seleccionada para validación

La vulnerabilidad elegida para esta fase fue:

V-01: Acceso anónimo al servicio FTP (vsftpd) con tráfico sin cifrado

La selección de esta vulnerabilidad se justificó por los siguientes motivos:

- Se trata de una debilidad distinta del vector principal de acceso ya demostrado en la fase forense, centrado en SSH;
- Fue confirmada externamente mediante Nmap y scripts NSE;
- Presentaba una configuración local crítica en vsftpd.conf;
- Su impacto potencial es elevado y fácilmente justificable;
- Su remediación es directa y verificable mediante reescaneo posterior.

Contexto técnico de la vulnerabilidad

Durante las fases anteriores del proyecto se confirmó que el servicio FTP del servidor objetivo:

- Se encontraba activo y habilitado al arranque;
- Estaba expuesto en el puerto 21/tcp;
- Respondía como vsftpd 3.0.3;
- Permitía autenticación anónima;
- Operaba con tráfico en texto plano;
- Según la configuración local observada, tenía habilitada la capacidad de escritura.

La configuración relevante observada en el sistema fue:

- anonymous_enable=YES
- local_enable=YES
- write_enable=YES
- ssl_enable=NO

Esta combinación representa una debilidad severa, ya que elimina controles básicos de autenticación y confidencialidad sobre un servicio históricamente sensible.

Validación técnica inicial

Enumeración externa con Nmap

La validación comenzó con una comprobación externa del servicio FTP mediante Nmap, utilizando scripts NSE orientados a confirmar acceso anónimo y comportamiento general del servidor.

Comando empleado:

$ sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33

Guardado en archivo:

$ sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33 -oN nmap_ftp_21.txt

Resultado observado

La salida confirmó:

- Puerto 21/tcp abierto;
- Servicio vsftpd 3.0.3;
- Autenticación anónima permitida;
- Conexión de control y de datos en texto plano.

Interpretación

Esta prueba fue suficiente para demostrar que el servicio FTP no solo estaba expuesto, sino que además podía ser accedido por un usuario no autenticado. Desde el punto de vista del pentesting, esto valida la vulnerabilidad a nivel de red sin necesidad inicial de realizar acciones intrusivas adicionales.

Explotación controlada

Conexión manual al servicio

Una vez validado el acceso anónimo con Nmap, se procedió a establecer una conexión manual al servicio FTP con el fin de confirmar interactividad, enumerar el comportamiento del servidor y, si el laboratorio lo permitía, comprobar de forma controlada la capacidad de escritura mediante un archivo marcador no ejecutable.

Comando empleado:

```
$ ftp 192.168.1.33
```

Credenciales utilizadas:

Name: anonymous
Password: anonymous

Comandos de interacción básica

Una vez iniciada la sesión, se utilizaron comandos básicos de enumeración:

pwd
ls
dir
cd /
bye

Resultado esperado de validación mínima

La explotación controlada se considera validada con resultado positivo si se observa:

- Autenticación anónima exitosa;
- Establecimiento de sesión en el servicio;
- Capacidad de listar contenido o interactuar con el servidor.

En el contexto del laboratorio, este nivel de prueba ya resulta suficiente para demostrar que existe una exposición crítica del servicio, aun sin necesidad de alterar contenido.

Comprobación controlada de escritura

Finalidad

Dado que la configuración local del servicio mostraba write_enable=YES, la siguiente prueba razonable consistió en verificar si el acceso anónimo podía además escribir o subir contenido al servidor. Esta prueba debe realizarse únicamente con un archivo benigno, de texto plano y no ejecutable, con el único propósito de confirmar la capacidad de modificación del servicio.

Preparación del archivo marcador

En la máquina Kali Linux se puede crear un archivo de prueba controlado:

echo "Prueba controlada de validacion FTP - laboratorio" > pentest_validacion_ftp.txt

Subida del archivo en sesión FTP

Dentro de la sesión FTP:

```
put pentest_validacion_ftp.txt
ls
bye
```

Interpretación

Existen dos escenarios posibles:

La subida es aceptada

Esto demostraría que el servicio FTP no solo permite acceso anónimo, sino también modificación de contenido, lo que eleva aún más su criticidad al posibilitar carga arbitraria de archivos.

La subida es rechazada

Aun en ese caso, la vulnerabilidad seguiría siendo grave, ya que el acceso anónimo y el tráfico sin cifrado continuarían representando una exposición crítica, aunque el impacto sobre integridad fuera menor de lo inicialmente esperado.

Impacto técnico de la vulnerabilidad

La validación de esta vulnerabilidad permite establecer el siguiente impacto potencial:

Sobre la confidencialidad

- Acceso no autenticado a contenido servido o almacenado mediante FTP;
- Posibilidad de enumerar estructura o recursos accesibles;
- Exposición de tráfico y credenciales en texto plano.

Sobre la integridad

- Si la escritura está efectivamente operativa, posibilidad de subida, modificación o sustitución de archivos;
- Riesgo de alteración del contenido publicado o transferido por el servidor.

Sobre la disponibilidad

- Posibilidad de abuso del servicio como canal no controlado de transferencia;
- Incremento del riesgo operacional por exposición de un servicio innecesario y débilmente protegido.

Sobre la superficie de ataque

- Existencia de un punto de entrada alternativo al servicio SSH ya comprometido;
- Ampliación innecesaria del perímetro accesible del host;
- Mayor facilidad de reconocimiento por parte de un atacante.

Severidad de la explotación validada

A la vista de la evidencia reunida, la vulnerabilidad se mantiene clasificada como:

Crítica

La razón principal es que combina varios factores de alto riesgo:

- Servicio expuesto en red;
- Acceso anónimo confirmado;
- Ausencia de cifrado;
- Configuración local que sugiere capacidad de escritura.

Incluso en ausencia de una manipulación efectiva demostrada del contenido, el simple hecho de aceptar autenticación anónima sobre FTP en un servidor comprometido y con exposición externa ya constituye una desviación severa respecto de cualquier práctica aceptable de seguridad.

MEDIDAS CORRECTIVAS APLICADAS

Objetivo de la remediación

Una vez validada la vulnerabilidad secundaria seleccionada, se procedió a definir y aplicar un conjunto de medidas correctivas orientadas a eliminar o reducir de forma significativa la exposición del servicio afectado.

En este caso, la prioridad principal fue neutralizar la debilidad crítica asociada al servicio FTP (vsftpd), sin perder de vista el contexto general del servidor, que ya presentaba otras configuraciones inseguras relevantes en SSH, WordPress y permisos del entorno web.

El propósito de esta fase fue doble:

- Corregir de forma inmediata la vulnerabilidad validada durante el pentesting;
- Reforzar el sistema para evitar que el mismo patrón de exposición se reproduzca a corto plazo.

Principio de corrección adoptado

Dado que el servicio FTP no aportaba un valor operativo imprescindible al rol real del servidor y suponía una superficie de ataque injustificada, la estrategia de corrección adoptada no consistió en “mejorar ligeramente” su configuración, sino en eliminar su exposición y retirar el servicio del conjunto de componentes activos del sistema.

Este criterio resulta coherente con el principio de mínimo servicio y con la arquitectura de red propuesta en la fase de rediseño, donde la transferencia de archivos insegura no forma parte de los servicios permitidos en producción.

Medidas correctivas sobre FTP

Detención inmediata del servicio

La primera acción correctiva consistió en detener el servicio FTP para eliminar su disponibilidad inmediata desde red.

Comando aplicado:

$ sudo systemctl stop vsftpd

Deshabilitación del arranque automático

Posteriormente, se deshabilitó el servicio para evitar que volviera a iniciarse automáticamente tras un reinicio del sistema.

Comando aplicado:

$ sudo systemctl disable vsftpd

Verificación del estado del servicio

Una vez detenida y deshabilitada la unidad, se verificó su estado para confirmar que el daemon dejaba de estar operativo.

Comando de validación:

$ sudo systemctl status vsftpd

Eliminación de exposición del puerto 21

Además de detener el servicio, se contempló como medida complementaria el bloqueo del puerto 21/tcp, ya fuera mediante firewall local o como parte del endurecimiento de la nueva arquitectura segmentada.

Comandos representativos:

$ sudo ufw deny 21/tcp
$ sudo ss -tulpn | grep :21

En caso de no utilizar ufw, esta medida puede implementarse mediante iptables, nftables o a nivel de firewall perimetral en la arquitectura propuesta.

Justificación técnica

Esta decisión se consideró la más adecuada por las siguientes razones:

- El servicio permitía autenticación anónima;
- Operaba sin cifrado;
- Su configuración local sugería capacidad de escritura;
- No era necesario para la prestación principal del servicio web;
- Representaba un punto de entrada alternativo e innecesario en un sistema ya comprometido.

Medidas complementarias asociadas a la superficie de ataque

Aunque la remediación principal de esta fase se centró en FTP, el contexto del sistema aconsejaba aplicar medidas complementarias sobre otros hallazgos críticos o de alto riesgo identificados en el análisis previo.

Endurecimiento del acceso SSH

Dado que la fase forense confirmó un acceso exitoso por SSH como root, se estableció como medida correctiva obligatoria el endurecimiento del servicio.

Ajustes recomendados en sshd_config:

- PermitRootLogin no
- PasswordAuthentication no *(cuando se disponga de acceso por clave pública)*
- AllowUsers debian
- MaxAuthTries 3
- LoginGraceTime 30

Comandos representativos:

```
$ sudo nano /etc/ssh/sshd_config
$ sudo systemctl restart ssh
$ sudo systemctl status ssh
```

Rotación de credenciales

Se consideró imprescindible el cambio de credenciales asociadas al sistema y a la aplicación, especialmente:

- Contraseña del usuario root
- Contraseña del usuario administrativo principal
- Contraseña del usuario de base de datos WordPress

Esta medida responde al hecho de que el acceso inicial ya se materializó mediante autenticación por contraseña.

Corrección de wp-config.php

El archivo wp-config.php presentaba varias debilidades graves, por lo que su remediación se incluyó como medida correctiva complementaria:

- Sustitución de DB_PASSWORD='123456' por una contraseña robusta;
- Regeneración de salts y claves de autenticación;
- Corrección de permisos del archivo.

Comandos representativos:

$ sudo chmod 640 /var/www/html/wp-config.php
$ sudo chown www-data:www-data /var/www/html/wp-config.php

El valor exacto de permisos puede variar según la política del laboratorio, pero en todo caso debe eliminarse el estado 777.

Corrección de permisos en .htaccess

Se corrigieron o se planificó corregir los permisos inseguros observados en los archivos .htaccess, que se encontraban con permisos 777.

Comando representativo:

$ sudo chmod 644 /var/www/html/.htaccess
$ sudo chmod 644 /var/www/html/wp-content/plugins/akismet/.htaccess

Reducción de servicios innecesarios

Como medida de hardening adicional, se estableció la conveniencia de deshabilitar servicios no esenciales para un servidor con función web, entre ellos:

- avahi-daemon
- bluetooth
- cups
- lightdm
- ModemManager
- wpa_supplicant

Esta medida no responde directamente a una explotación demostrada, pero sí reduce la superficie general de ataque y alinea el host con una configuración más propia de servidor que de estación de trabajo.

Medidas correctivas a nivel de arquitectura

Las acciones sobre el host se complementan con las medidas de diseño de red definidas en la topología propuesta:

- Ubicación del servidor en una DMZ;
- Separación entre inside, outside y DMZ mediante firewall;
- Bloqueo de acceso desde la red externa a servicios no publicados;
- Eliminación de FTP del conjunto de servicios permitidos;
- Restricción del acceso administrativo al servidor desde la red interna autorizada.

Estas medidas refuerzan la corrección local del sistema al impedir que una configuración insegura vuelva a quedar expuesta sin control perimetral.

Resultado esperado tras la remediación

Una vez aplicadas las medidas correctivas, el sistema debería presentar el siguiente comportamiento esperado:

- El puerto 21/tcp ya no debe aparecer abierto ni accesible desde red;
- vsftpd debe permanecer detenido y deshabilitado;
- SSH debe seguir operativo solo bajo configuración endurecida;
- WordPress debe mantener funcionalidad, pero con credenciales robustas y permisos corregidos;
- El entorno web debe dejar de exponer archivos o configuraciones innecesariamente permisivas;
- La superficie de ataque general del host debe reducirse de forma apreciable.

Eliminacion del usuario “sospechoso”

Durante la revisión de MariaDB se identificó la cuenta **user@localhost**, no autorizada, configurada con privilegios globales administrativos mediante **ALL PRIVILEGES ON *.* WITH GRANT OPTION**. Este hallazgo fue tratado como un mecanismo de persistencia lógica y un riesgo crítico de control total sobre el motor de base de datos.

Como medida de erradicación, se verificaron previamente sus privilegios efectivos, se procedió a la eliminación de la cuenta mediante **DROP USER 'user'@'localhost';.**

Se validó posteriormente que el usuario ya no se encontraba presente en el sistema gestor de bases de datos.

Esta acción formó parte de la fase de contención y erradicación, con el objetivo de eliminar accesos administrativos no legítimos y reducir la posibilidad de reutilización de privilegios tras el compromiso del servidor.

REVALIDACIÓN Y VERIFICACIÓN POSTERIOR

Objetivo de la revalidación

Una vez aplicadas las medidas correctivas sobre la vulnerabilidad secundaria validada y sobre otras debilidades críticas del sistema, se llevó a cabo una fase de revalidación técnica con el fin de comprobar que la exposición previamente identificada había sido reducida o eliminada de forma efectiva.

El objetivo principal de esta etapa fue contrastar el estado del servidor antes y después de la remediación, verificando especialmente:

- la desaparición del servicio FTP como punto de exposición en red;
- el endurecimiento del acceso administrativo;
- la reducción de la superficie de ataque;
- y la permanencia de la funcionalidad básica del servidor en aquellos servicios que debían continuar operativos.

Esta fase es esencial dentro de un informe de pentesting, ya que una medida correctiva no debe considerarse suficiente por el mero hecho de haberse aplicado, sino únicamente cuando existe evidencia técnica de que ha mitigado el riesgo identificado.

Metodología de verificación

La revalidación se efectuó desde dos perspectivas complementarias:

Verificación local en el host

Se revisó el estado de los servicios, los puertos en escucha y determinadas configuraciones del sistema para confirmar que las acciones de remediación habían sido aplicadas correctamente desde el punto de vista interno.

Verificación externa desde Kali Linux

Se repitieron escaneos de red y comprobaciones dirigidas contra el servidor objetivo con el fin de validar que los cambios tenían efecto real sobre la superficie de ataque observable desde otro equipo del laboratorio.

Este enfoque permitió comparar la corrección lógica y administrativa del servicio con su impacto efectivo en términos de exposición externa.

Verificación local del estado del servicio FTP

La primera comprobación tras la remediación consistió en validar que el servicio vsftpd se encontraba detenido y deshabilitado.

Comandos representativos:

$ sudo systemctl status vsftpd
$ sudo systemctl is-enabled vsftpd

Resultado esperado

- vsftpd en estado inactive o dead
- servicio disabled

Interpretación

Si el servicio deja de estar activo y ya no arranca automáticamente con el sistema, la remediación habrá sido aplicada correctamente a nivel de servicio. No obstante, esta verificación local no es suficiente por sí sola, ya que aún sería necesario confirmar que el puerto asociado ha desaparecido de la exposición en red.

Verificación local de puertos en escucha

Con el fin de confirmar la desaparición efectiva del puerto 21/tcp, se revisó la lista de sockets en escucha del sistema.

Comandos representativos:

$ ss -tulpn
$ sudo netstat -tulpn

Resultado esperado

- ausencia de 21/tcp
- permanencia únicamente de servicios justificados, como:
  - 22/tcp para SSH
  - 80/tcp para HTTP
- 3306/tcp restringido a 127.0.0.1 si MariaDB se mantiene solo en local

Interpretación

La desaparición del puerto 21/tcp en la revisión local confirma que el daemon ya no está enlazado a la red. Este paso constituye una validación necesaria del cierre del vector secundario identificado durante el pentesting.

Reescaneo externo con Nmap

Tras la validación local, se repitió el reconocimiento de puertos desde la máquina Kali Linux con el fin de confirmar que el servicio FTP ya no resultaba visible desde la red.

Escaneo general de puertos

Comando representativo:

$ sudo nmap -sS -sV -Pn -p- 192.168.1.33

Guardado de resultados:

$ sudo nmap -sS -sV -Pn -p- 192.168.1.33 -oA nmap_full_post_remediation

Resultado esperado

- 21/tcp en estado closed o filtered
- 22/tcp y 80/tcp como servicios todavía visibles, si se mantienen operativos
- ausencia de nuevos puertos no justificados

Interpretación

Si el puerto 21 deja de aparecer como abierto, puede considerarse validada la mitigación efectiva de la vulnerabilidad secundaria a nivel de exposición de red.

Escaneo focalizado de los servicios principales

Comando representativo:

sudo nmap -sV -Pn -p 21,22,80 192.168.1.33

Guardado de resultados:

$ sudo nmap -sV -Pn -p 21,22,80 192.168.1.33 -oN nmap_post_21_22_80.txt

Resultado esperado

- 21/tcp ya no accesible
- 22/tcp visible únicamente como servicio administrativo endurecido
- 80/tcp visible para la prestación del servicio web

Interpretación

Este escaneo permite mostrar de forma clara y directa la reducción de la superficie de ataque sobre los puertos que inicialmente habían sido identificados como más relevantes.

Revalidación específica del servicio FTP

Para confirmar que la vulnerabilidad secundaria había desaparecido también a nivel funcional, se puede repetir el escaneo con scripts NSE específicos sobre el puerto 21.

Comando representativo:

sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33

Resultado esperado

- El script ya no debe devolver:
    - Anonymous FTP login allowed
- El puerto no debe responder

Interpretación

La desaparición de este resultado constituye una evidencia especialmente fuerte de que la vulnerabilidad validada durante el pentesting ha sido mitigada con éxito.

Revalidación del acceso SSH

Dado que SSH fue el vector de acceso explotado durante el incidente, y aunque no constituya la vulnerabilidad secundaria del pentest, resulta metodológicamente adecuado verificar que las medidas de endurecimiento sobre este servicio hayan reducido el riesgo asociado.

Comandos representativos:

$ sudo grep -E "PermitRootLogin|PasswordAuthentication|AllowUsers|MaxAuthTries" /etc/ssh/sshd_config
$ sudo systemctl status ssh
$ sudo nmap -sV -Pn -p 22 192.168.1.33

Resultado esperado

- PermitRootLogin no
- reducción de superficie administrativa
- mantenimiento del servicio únicamente para usuarios autorizados

Interpretación

Estas comprobaciones permiten mostrar que, además de eliminar FTP como vector secundario, el sistema ha comenzado a corregir también el punto principal de exposición evidenciado en la fase forense.

Revalidación del entorno web

Aunque la vulnerabilidad secundaria seleccionada fue FTP, resulta conveniente verificar que el entorno web sigue funcionando y que la corrección de permisos o configuraciones no ha provocado una degradación del servicio.

Comandos representativos:

$ curl -I http://192.168.1.33
$ curl -I http://192.168.1.33/wp-login.php
$ curl -I http://192.168.1.33/wp-json

Resultado esperado

- respuesta HTTP válida
- disponibilidad del servicio web
- persistencia del sitio sin exponer innecesariamente configuraciones inseguras adicionales

Interpretación

Este paso garantiza que la remediación no solo reduce riesgo, sino que preserva la funcionalidad básica del servidor.

Comparativa antes / después

La revalidación posterior debe permitir presentar una comparación sencilla entre el estado inicial y el estado posterior a la remediación.

Estado previo

- 21/tcp abierto y accesible
- vsftpd activo y habilitado
- autenticación anónima confirmada
- tráfico FTP en texto plano
- superficie de ataque ampliada

Estado posterior esperado

- vsftpd detenido y deshabilitado
- 21/tcp ausente o filtrado

- desaparición de acceso anónimo al servicio
- reducción visible de la superficie de ataque
- mantenimiento controlado de los servicios necesarios

Interpretación

Esta comparación constituye la evidencia central de que la remediación aplicada no ha sido meramente declarativa, sino efectiva y verificable.

CONCLUSIONES

La fase de pentesting realizada sobre el servidor Debian GNU/Linux 12 permitió confirmar que el sistema mantenía una superficie de ataque excesiva y técnicamente injustificada, coherente con el estado de inseguridad previamente identificado durante la fase forense.

La evaluación activa desde Kali Linux validó externamente la exposición de varios servicios relevantes, entre ellos FTP, SSH y HTTP, así como diversas debilidades asociadas al entorno web y a la configuración general del host.

Desde una perspectiva global, el hallazgo más importante del proyecto sigue siendo la confirmación de que el acceso inicial al sistema se produjo a través de SSH, mediante autenticación por contraseña del usuario root, tal y como quedó evidenciado en los registros del servicio.

Este hecho permitió establecer con claridad el vector de compromiso principal y justificar la necesidad de endurecer de forma prioritaria el acceso remoto administrativo. No obstante, el pentesting demostró además que el sistema no presentaba una única debilidad aislada, sino un conjunto de configuraciones inseguras concurrentes capaces de facilitar accesos adicionales, enumeración sensible o exposición innecesaria de servicios.

En ese contexto, la vulnerabilidad secundaria seleccionada para validación controlada fue la exposición del servicio FTP (vsftpd), que se encontraba accesible en el puerto 21/tcp, permitía autenticación anónima y operaba con tráfico en texto plano.

Esta debilidad quedó validada de forma reproducible mediante Nmap y scripts NSE, lo que permitió clasificarla como una vulnerabilidad de severidad crítica, distinta del acceso principal ya documentado por la investigación forense. Su relevancia técnica se vio reforzada por la revisión local de vsftpd.conf, donde se confirmó además la presencia de parámetros inseguros como acceso anónimo, escritura habilitada y ausencia de cifrado.

Junto a ello, la fase de enumeración también puso de manifiesto otras debilidades significativas. El servicio HTTP revelaba información del servidor mediante banners y mantenía expuestos recursos propios de WordPress, como wp-login.php, wp-json y readme.html, lo que facilitaba tareas de fingerprinting y enumeración.

Asimismo, la revisión local del entorno WordPress confirmó un wp-config.php con contraseña de base de datos débil, claves criptográficas por defecto y permisos excesivamente permisivos, además de archivos .htaccess también configurados con permisos 777. Aunque estas condiciones no demostraron por sí mismas una explotación web remota concluyente, sí evidenciaron un estado de hardening claramente insuficiente del entorno web.

Por otra parte, la búsqueda orientada a detectar rootkits, malware residente o mecanismos de persistencia no permitió identificar evidencias concluyentes de un implante activo sofisticado. No se localizaron authorized_keys en cuentas privilegiadas o de usuario, no se observaron cron jobs claramente maliciosos, ni se detectaron binarios SUID/SGID anómalos en rutas impropias del sistema.

Tampoco se hallaron ejecutables sospechosos en directorios temporales o un uso anómalo de /etc/ld.so.preload. Del mismo modo, la revisión del webroot no reveló una web shell inequívoca ni archivos ocultos claramente maliciosos, aunque sí confirmó la presencia de múltiples configuraciones inseguras. En consecuencia, la evidencia disponible apunta con mayor fuerza a un compromiso facilitado por debilidades de configuración y exposición de servicios, más que a la implantación demostrada de malware persistente.

Las medidas correctivas definidas y aplicadas se orientaron a reducir la superficie de ataque, eliminar el servicio FTP del conjunto de componentes expuestos, endurecer el servicio SSH, corregir credenciales y permisos inseguros del entorno WordPress y avanzar hacia una arquitectura más segura basada en segmentación, firewall y DMZ.

En este sentido, la remediación no se limitó a “cerrar un puerto”, sino que se integró dentro de una estrategia más amplia de saneamiento del sistema y mejora de su postura de seguridad.

En conclusión, el pentesting permitió demostrar que el servidor no solo había sido comprometido previamente mediante un servicio remoto mal endurecido, sino que además conservaba debilidades adicionales de criticidad elevada que podían haber sido utilizadas como vectores alternativos o complementarios de acceso.

La validación de la vulnerabilidad secundaria y la posterior definición de medidas correctivas aportan una base técnica sólida para justificar el endurecimiento integral del sistema.

Por tanto, esta fase del proyecto no solo confirma la existencia de vulnerabilidades, sino que aporta evidencia útil para su priorización, corrección y verificación posterior, integrándose de manera coherente con la investigación forense y con la propuesta de red segura elaborada en etapas anteriores.

ANEXOS

V-00. Vulnerabilidad explotada: SSH inseguro con acceso root

Desde el servidor Debian

Revisamos configuración SSH

```
debian@debian:~$ sudo grep -E "PermitRootLogin|PasswordAutentication|AllowUser|MaxAuthTri
es|LoginGraceTime" /etc/ssh/sshd_config
#LoginGraceTime 2m
PermitRootLogin yes
#MaxAuthTries 6
# the setting of "PermitRootLogin prohibit-password".
```

```
debian@debian:~$ sudo cat /etc/ssh/sshd_config

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin yes
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes
```

```
# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile     .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

#AuthorizedKeysCommand none
#AuthorizedKeysCommandUser nobody

# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
#HostbasedAuthentication no
# Change to yes if you don't trust ~/.ssh/known_hosts for
# HostbasedAuthentication
#IgnoreUserKnownHosts no
# Don't read the user's ~/.rhosts and ~/.shosts files
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to no here!
PasswordAuthentication yes
#PermitEmptyPasswords no

# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
KbdInteractiveAuthentication no

# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes
#KerberosTicketCleanup yes
#KerberosGetAFSToken no

# GSSAPI options
#GSSAPIAuthentication no
#GSSAPICleanupCredentials yes
#GSSAPIStrictAcceptorCheck yes
#GSSAPIKeyExchange no
```

```
# Set this to 'yes' to enable PAM authentication, account processing,
# and session processing. If this is enabled, PAM authentication will
# be allowed through the KbdInteractiveAuthentication and
# PasswordAuthentication.  Depending on your PAM configuration,
# PAM authentication via KbdInteractiveAuthentication may bypass
# the setting of "PermitRootLogin prohibit-password".
# If you just want the PAM account and session checks to run without
# PAM authentication, then enable this but set PasswordAuthentication
# and KbdInteractiveAuthentication to 'no'.
UsePAM yes

#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
#Banner none

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*
```

```
# override default of no subsystems
Subsystem       sftp    /usr/lib/openssh/sftp-server

# Example of overriding settings on a per-user basis
#Match User anoncvs
#       X11Forwarding no
#       AllowTcpForwarding no
#       PermitTTY no
#       ForceCommand cvs server
```

Comprobacion de logs de acceso

```
debian@debian:~$ sudo journalctl -u ssh --since "2024-10-08 17:40:00" --until "2024-10-08 17:42:00" --no-pager
Oct 08 17:40:59 debian sshd[1650]: Accepted password for root from 192.168.0.134 port 45623 ssh2
Oct 08 17:40:59 debian sshd[1650]: pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
Oct 08 17:40:59 debian sshd[1650]: pam_env(sshd:session): deprecated reading of user environment enabled
```

```
debian@debian:~$ sudo journalctl -u ssh --no-pager | grep -Ei "Accepted|session opened|session closed"
Oct 08 17:40:59 debian sshd[1650]: Accepted password for root from 192.168.0.134 port 45623 ssh2
Oct 08 17:40:59 debian sshd[1650]: pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
```

Historial del Root

```
debian@debian:~$ sudo ls -l /root/.bash_history
-rw------- 1 root root 127 Jul 31  2024 /root/.bash_history
debian@debian:~$ sudo cat /root/.bash_history
sudo visudo
sudo systemctl stop speech-dispatcher
sudo systemctl disable speech-dispatcher
systemctl list-units --type=service
```

Desde Kali Linux al Servidor Debian

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -Pn -p 22 192.168.1.33 -oN ~/Desktop/pentest_resultados/20_nmap_ssh_pre.txt
sudo: unable to resolve host kali: Nombre o servicio desconocido
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 10:43 +0100
Nmap scan report for 192.168.1.33
Host is up (0.00085s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
MAC Address: 08:00:27:8C:8C:A0 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.85 seconds
```

```
└─$ sudo nmap -sV -Pn -p 22 --script ssh2-enum-algos 192.168.1.33 -oN ~/Desktop/pentest_resultados/21_nmap_ssh_algos_pre.txt
sudo: unable to resolve host kali: Nombre o servicio desconocido
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 10:44 +0100
Nmap scan report for 192.168.1.33
Host is up (0.00074s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
| ssh2-enum-algos:
|   kex_algorithms: (11)
|       sntrup761x25519-sha512@openssh.com
|       curve25519-sha256
|       curve25519-sha256@libssh.org
|       ecdh-sha2-nistp256
|       ecdh-sha2-nistp384
|       ecdh-sha2-nistp521
|       diffie-hellman-group-exchange-sha256
|       diffie-hellman-group16-sha512
|       diffie-hellman-group18-sha512
|       diffie-hellman-group14-sha256
|       kex-strict-s-v00@openssh.com
|   server_host_key_algorithms: (4)
|       rsa-sha2-512
|       rsa-sha2-256
|       ecdsa-sha2-nistp256
|       ssh-ed25519
|   encryption_algorithms: (6)
|       chacha20-poly1305@openssh.com
|       aes128-ctr
|       aes192-ctr
|       aes256-ctr
|       aes128-gcm@openssh.com
|       aes256-gcm@openssh.com
|   mac_algorithms: (10)
|       umac-64-etm@openssh.com
|       umac-128-etm@openssh.com
|       hmac-sha2-256-etm@openssh.com
|       hmac-sha2-512-etm@openssh.com
|       hmac-sha1-etm@openssh.com
|       umac-64@openssh.com
|       umac-128@openssh.com
|       hmac-sha2-256
|       hmac-sha2-512
|       hmac-sha1
|   compression_algorithms: (2)
|       none
|_      zlib@openssh.com
MAC Address: 08:00:27:8C:8C:A0 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.80 seconds
```

Comprobamos manualmente que podemos acceder

```
┌──(kali㉿kali)-[~]
└─$ ssh root@192.168.1.33
The authenticity of host '192.168.1.33 (192.168.1.33)' can't be established.
ED25519 key fingerprint is: SHA256:y+azUUsJLjX3WV8+EjMaTB4WybvwH7XBLCt7vp3zvLg
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.1.33' (ED25519) to the list of known hosts.
root@192.168.1.33's password:
Linux debian 6.1.0-25-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.106-3 (2024-08-26) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@debian:~#
```

Corregimos el fallo en Debian

Creamos un Backup del archivo original

```
debian@debian:~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak.$(date +%F_%H%M%S)
```

Creamos Hardening para SSH

```
debian@debian:~$ sudo tee /etc/ssh/sshd_config.d/99-hardening.conf > /dev/null <<'EOF'
> PermitRootLogin no
> MaxAuthTries 3
> LoginGraceTime 30
> AllowUsers debian
> EOF
```

Verificamos configuración

```
debian@debian:~$ sudo sshd -t
debian@debian:~$
```

Reiniciamos el servicio y verificamos que el servicio este activo

```
debian@debian:~$ sudo systemctl restart ssh
debian@debian:~$ sudo systemctl status ssh --no-pager
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-03-17 06:21:05 EDT; 13s ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 5355 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 5357 (sshd)
      Tasks: 1 (limit: 9427)
     Memory: 1.4M
        CPU: 16ms
     CGroup: /system.slice/ssh.service
             └─5357 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Mar 17 06:21:05 debian systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Mar 17 06:21:05 debian sshd[5357]: Server listening on 0.0.0.0 port 22.
Mar 17 06:21:05 debian sshd[5357]: Server listening on :: port 22.
Mar 17 06:21:05 debian systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

Desde Kali comprobamos los cambios y la corrección.

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -Pn -p 22 192.168.1.33 -oN ~/Desktop/pentest_resultados/20_nmap_ssh_post.txt
sudo: unable to resolve host kali: Nombre o servicio desconocido
[sudo] contraseña para kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 11:28 +0100
Nmap scan report for 192.168.1.33
Host is up (0.00063s latency).

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
MAC Address: 08:00:27:8C:8C:A0 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.78 seconds
```

No hay Accesso como root

```
┌──(kali㉿kali)-[~]
└─$ ssh root@192.168.1.33
root@192.168.1.33's password:
Permission denied, please try again.
root@192.168.1.33's password:
Permission denied, please try again.
root@192.168.1.33's password:
Received disconnect from 192.168.1.33 port 22:2: Too many authentication failures
Disconnected from 192.168.1.33 port 22
```

Hay Accesso como usuario

```
┌──(kali㉿kali)-[~]
└─$ ssh debian@192.168.1.33
debian@192.168.1.33's password:
Linux debian 6.1.0-25-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.106-3 (2024-08-26) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
debian@debian:~$ exito
-bash: exito: command not found
debian@debian:~$ exit
logout
Connection to 192.168.1.33 closed.
```

V-01. Vulnerabilidad secundaria: FTP anónimo y sin cifrado.

Explotada de forma controlada

En el Servidor Debian

Verificamos la situación del servicio.

```
debian@debian:~$ systemctl status vsftpd --no-pager
● vsftpd.service - vsftpd FTP server
     Loaded: loaded (/lib/systemd/system/vsftpd.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-03-17 04:39:25 EDT; 1h 55min ago
    Process: 607 ExecStartPre=/bin/mkdir -p /var/run/vsftpd/empty (code=exited, status=0/SUCCESS)
   Main PID: 609 (vsftpd)
      Tasks: 1 (limit: 9427)
     Memory: 1.0M
        CPU: 14ms
     CGroup: /system.slice/vsftpd.service
             └─609 /usr/sbin/vsftpd /etc/vsftpd.conf

Warning: some journal files were not opened due to insufficient permissions.
```

```
debian@debian:~$ systemctl is-enabled vsftpd
enabled
```

Revisamos la configuración

```
debian@debian:~$ sudo grep -E "anonymous_enable|local_enable|write_enable@ssl_enable" /etc/vsftpd.conf
[sudo] password for debian:
anonymous_enable=YES
local_enable=YES
```

```
debian@debian:~$ sudo grep -E "anonymous_enable|local_enable|write_enable@ssl_enable" /etc/vsftpd.conf
[sudo] password for debian:
anonymous_enable=YES
local_enable=YES
debian@debian:~$ sudo cat /etc/vsftpd.conf
# Example config file /etc/vsftpd.conf
#
# The default compiled in settings are fairly paranoid. This sample file
# loosens things up a bit, to make the ftp daemon more usable.
# Please see vsftpd.conf.5 for all compiled in defaults.
#
# READ THIS: This example file is NOT an exhaustive list of vsftpd options.
# Please read the vsftpd.conf.5 manual page to get a full idea of vsftpd's
# capabilities.
#
#
# Run standalone?  vsftpd can run either from an inetd or as a standalone
# daemon started from an initscript.
listen=NO
#
# This directive enables listening on IPv6 sockets. By default, listening
# on the IPv6 "any" address (::) will accept connections from both IPv6
# and IPv4 clients. It is not necessary to listen on "both" IPv4 and IPv6
# sockets. If you want that (perhaps because you want to listen on specific
# addresses) then you must run two copies of vsftpd with two configuration
# files.
listen_ipv6=YES
#
# Allow anonymous FTP? (Disabled by default).
anonymous_enable=YES
#
# Uncomment this to allow local users to log in.
local_enable=YES
#
# Uncomment this to enable any form of FTP write command.
write_enable=YES
#
```

```
# Default umask for local users is 077. You may wish to change this to 022,
# if your users expect that (022 is used by most other ftpd's)
#local_umask=022
#
# Uncomment this to allow the anonymous FTP user to upload files. This only
# has an effect if the above global write enable is activated. Also, you will
# obviously need to create a directory writable by the FTP user.
#anon_upload_enable=YES
#
# Uncomment this if you want the anonymous FTP user to be able to create
# new directories.
#anon_mkdir_write_enable=YES
#
# Activate directory messages - messages given to remote users when they
# go into a certain directory.
dirmessage_enable=YES
#
# If enabled, vsftpd will display directory listings with the time
# in  your  local  time  zone.  The default is to display GMT. The
# times returned by the MDTM FTP command are also affected by this
# option.
use_localtime=YES
#
# Activate logging of uploads/downloads.
xferlog_enable=YES
#
# Make sure PORT transfer connections originate from port 20 (ftp-data).
connect_from_port_20=YES
#
# If you want, you can arrange for uploaded anonymous files to be owned by
# a different user. Note! Using "root" for uploaded files is not
# recommended!
#chown_uploads=YES
#chown_username=whoever
#
# You may override where the log file goes if you like. The default is shown
# below.
#xferlog_file=/var/log/vsftpd.log
```

```
# If you want, you can have your log file in standard ftpd xferlog format.
# Note that the default log file location is /var/log/xferlog in this case.
#xferlog_std_format=YES
#
# You may change the default value for timing out an idle session.
#idle_session_timeout=600
#
# You may change the default value for timing out a data connection.
#data_connection_timeout=120
#
# It is recommended that you define on your system a unique user which the
# ftp server can use as a totally isolated and unprivileged user.
#nopriv_user=ftpsecure
#
# Enable this and the server will recognise asynchronous ABOR requests. Not
# recommended for security (the code is non-trivial). Not enabling it,
# however, may confuse older FTP clients.
#async_abor_enable=YES
#
# By default the server will pretend to allow ASCII mode but in fact ignore
# the request. Turn on the below options to have the server actually do ASCII
# mangling on files when in ASCII mode.
# Beware that on some FTP servers, ASCII support allows a denial of service
# attack (DoS) via the command "SIZE /big/file" in ASCII mode. vsftpd
# predicted this attack and has always been safe, reporting the size of the
# raw file.
# ASCII mangling is a horrible feature of the protocol.
#ascii_upload_enable=YES
#ascii_download_enable=YES
#
# You may fully customise the login banner string:
#ftpd_banner=Welcome to blah FTP service.
#
# You may specify a file of disallowed anonymous e-mail addresses. Apparently
# useful for combatting certain DoS attacks.
#deny_email_enable=YES
# (default follows)
#banned_email_file=/etc/vsftpd.banned_emails
#
```

```
# You may restrict local users to their home directories.  See the FAQ for
# the possible risks in this before using chroot_local_user or
# chroot_list_enable below.
#chroot_local_user=YES
#
# You may specify an explicit list of local users to chroot() to their home
# directory. If chroot_local_user is YES, then this list becomes a list of
# users to NOT chroot().
# (Warning! chroot'ing can be very dangerous. If using chroot, make sure that
# the user does not have write access to the top level directory within the
# chroot)
#chroot_local_user=YES
#chroot_list_enable=YES
# (default follows)
#chroot_list_file=/etc/vsftpd.chroot_list
#
# You may activate the "-R" option to the builtin ls. This is disabled by
# default to avoid remote users being able to cause excessive I/O on large
# sites. However, some broken FTP clients such as "ncftp" and "mirror" assume
# the presence of the "-R" option, so there is a strong case for enabling it.
#ls_recurse_enable=YES
#
# Customization
#
# Some of vsftpd's settings don't fit the filesystem layout by
# default.
#
# This option should be the name of a directory which is empty.  Also, the
# directory should not be writable by the ftp user. This directory is used
# as a secure chroot() jail at times vsftpd does not require filesystem
# access.
secure_chroot_dir=/var/run/vsftpd/empty
#
# This string is the name of the PAM service vsftpd will use.
pam_service_name=vsftpd
#
# This option specifies the location of the RSA certificate to use for SSL
# encrypted connections.
rsa_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
rsa_private_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
```

```
ssl_enable=NO

#
# Uncomment this to indicate that vsftpd use a utf8 filesystem.
#utf8_filesystem=YES
```

Verificamos que el puerto está en escucha

```
debian@debian:~$ ss -tulpn | grep :21
tcp   LISTEN 0      32                 *:21               *:*
```

Revisamos logs

```
debian@debian:~$ sudo journalctl -u vsftpd --no-pager
Oct 08 16:09:01 debian systemd[1]: Starting vsftpd.service - vsftpd FTP server...
Oct 08 16:09:01 debian systemd[1]: Started vsftpd.service - vsftpd FTP server.
Oct 08 16:10:37 debian systemd[1]: Stopping vsftpd.service - vsftpd FTP server...
Oct 08 16:10:37 debian systemd[1]: vsftpd.service: Deactivated successfully.
Oct 08 16:10:37 debian systemd[1]: Stopped vsftpd.service - vsftpd FTP server.
Oct 08 16:10:37 debian systemd[1]: Starting vsftpd.service - vsftpd FTP server...
Oct 08 16:10:37 debian systemd[1]: Started vsftpd.service - vsftpd FTP server.
-- Boot 342683d8f35244b08c4f3863f2978eca --
Oct 08 16:43:18 debian systemd[1]: Starting vsftpd.service - vsftpd FTP server...
Oct 08 16:43:18 debian systemd[1]: Started vsftpd.service - vsftpd FTP server.
-- Boot d28e179bf5884b25bf94452c79fd0afa --
Oct 08 16:48:02 debian systemd[1]: Starting vsftpd.service - vsftpd FTP server...
Oct 08 16:48:02 debian systemd[1]: Started vsftpd.service - vsftpd FTP server.
-- Boot af0a79f76920440c8e08594d6547449b --
Oct 08 17:28:37 debian systemd[1]: Starting vsftpd.service - vsftpd FTP server...
Oct 08 17:28:37 debian systemd[1]: Started vsftpd.service - vsftpd FTP server.
```

Desde Kali Linuz comprobamos el puerto.

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33 -oN ~/Desktop/pentest_resultados/nmap_ftp_pre.txt
sudo: unable to resolve host kali: Nombre n servicio desconocido
[sudo] contraseña para kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 11:53 +0100
Nmap scan report for 192.168.1.33
Host is up (0.00072s latency).

PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.3
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to ::ffff:192.168.1.22
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      At session startup, client count was 2
|      vsFTPd 3.0.3 - secure, fast, stable
|_End of status
MAC Address: 08:00:27:8C:8C:A0 (Oracle VirtualBox virtual NIC)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.76 seconds
```

Comprobamos que podemos acceder de forma manual

(explotación)

```
┌──(kali㉿kali)-[~]
└─$ ftp 192.168.1.33
Connected to 192.168.1.33.
220 (vsFTPd 3.0.3)
Name (192.168.1.33:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||37030|)
150 Here comes the directory listing.
226 Directory send OK.
ftp> dir
229 Entering Extended Passive Mode (|||35398|)
150 Here comes the directory listing.
226 Directory send OK.
ftp> bye
221 Goodbye.
```

Corregimos el fallo en Debian

Paramos el servicio y deshabilitamos el arranque automático.

```
debian@debian:~$ sudo systemctl stop vsftpd
[sudo] password for debian:
debian@debian:~$ sudo systemctl disable vsftpd
Synchronizing state of vsftpd.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install disable vsftpd
Removed "/etc/systemd/system/multi-user.target.wants/vsftpd.service".
```

Fijamos las reglas el Firewall para los puertos 21 – 22 – 80

```
debian@debian:~$ sudo ufw deny 21/tcp
Rules updated
Rules updated (v6)
```

```
debian@debian:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
debian@debian:~$ sudo ufw allow 80/tcp
Rules updated
Rules updated (v6)
```

Activamos el Firewall

```
debian@debian:~$ sudo ufw enable
Firewall is active and enabled on system startup
```

Verificamos el status

```
debian@debian:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
21/tcp                     DENY IN     Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
21/tcp (v6)                DENY IN     Anywhere (v6)
```

```
debian@debian:~$ systemctl status vsftpd --no-pager
○ vsftpd.service - vsftpd FTP server
     Loaded: loaded (/lib/systemd/system/vsftpd.service; disabled; preset: enabled)
     Active: inactive (dead)
```

```
debian@debian:~$ systemctl is-enabled vsftpd
disabled
```

Verificamos desde Kali Linux

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -Pn -p 21 --script ftp-anon,ftp-syst 192.168.1.33 -oN ~/Desktop/pentest_resultados/41_nmap_ftp_post.txt
sudo: unable to resolve host kali: Nombre o servicio desconocido
[sudo] contraseña para kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 12:22 +0100
Nmap scan report for 192.168.1.33
Host is up (0.00063s latency).

PORT   STATE    SERVICE VERSION
21/tcp filtered ftp
MAC Address: 08:00:27:BC:8C:A0 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.93 seconds
```

Verificamos manualmente la conexion

```
┌──(kali㉿kali)-[~]
└─$ ftp 192.168.1.33



exit
^C
```

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sS -sV -Pn -p 21,22,80 192.168.1.33 -oN ~/Desktop/pentest_resultados/42_nmap_21_22_80_post.txt
sudo: unable to resolve host kali: Nombre o servicio desconocido
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 12:24 +0100
Nmap scan report for 192.168.1.33
Host is up (0.00051s latency).

PORT   STATE    SERVICE VERSION
21/tcp filtered ftp
22/tcp open     ssh     OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
80/tcp open     http    Apache httpd 2.4.62 ((Debian))
MAC Address: 08:00:27:BC:8C:A0 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.69 seconds
```

V-02 y V-03. WordPress inseguro: wp-config.php, credenciales débiles, salts por defecto, permisos 777. Hardening y exposición web.

Verificamos permisos

```
debian@debian:~$ sudo ls -l /var/www/html/wp-config.php
-rwxrwxrwx 1 www-data www-data 3017 Sep 30  2024 /var/www/html/wp-config.php
```

Verificamos contenido

```
debian@debian:~$ sudo cat /var/www/html/wp-config.php
<?php
/**
 * The base configuration for WordPress
 *
 * The wp-config.php creation script uses this file during the installation.
 * You don't have to use the website, you can copy this file to "wp-config.php"
 * and fill in the values.
 *
 * This file contains the following configurations:
 *
 * * Database settings
 * * Secret keys
 * * Database table prefix
 * * ABSPATH
 *
 * @link https://developer.wordpress.org/advanced-administration/wordpress/wp-config/
 *
 * @package WordPress
 */

// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress' );

/** Database username */
define( 'DB_USER', 'wordpressuser' );

/** Database password */
define( 'DB_PASSWORD', '123456' );

/** Database hostname */
define( 'DB_HOST', 'localhost' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );
```

```
/**#@+
 * Authentication unique keys and salts.
 *
 * Change these to different unique phrases! You can generate these using
 * the {@link https://api.wordpress.org/secret-key/1.1/salt/ WordPress.org secret-key service}.
 *
 * You can change these at any point in time to invalidate all existing cookies.
 * This will force all users to have to log in again.
 *
 * @since 2.6.0
 */
define( 'AUTH_KEY',         'put your unique phrase here' );
define( 'SECURE_AUTH_KEY',  'put your unique phrase here' );
define( 'LOGGED_IN_KEY',    'put your unique phrase here' );
define( 'NONCE_KEY',        'put your unique phrase here' );
define( 'AUTH_SALT',        'put your unique phrase here' );
define( 'SECURE_AUTH_SALT', 'put your unique phrase here' );
define( 'LOGGED_IN_SALT',   'put your unique phrase here' );
define( 'NONCE_SALT',       'put your unique phrase here' );

/**#@-*/

/**
 * WordPress database table prefix.
 *
 * You can have multiple installations in one database if you give each
 * a unique prefix. Only numbers, letters, and underscores please!
 */
$table_prefix = 'wp_';

/**
 * For developers: WordPress debugging mode.
 *
 * Change this to true to enable the display of notices during development.
 * It is strongly recommended that plugin and theme developers use WP_DEBUG
 * in their development environments.
 *
 * For information on other constants that can be used for debugging,
 * visit the documentation.
 *
```

```
 * @link https://developer.wordpress.org/advanced-administration/debug/debug-wordpress/
 */
define( 'WP_DEBUG', false );

/* Add any custom values between this line and the "stop editing" line. */



/* That's all, stop editing! Happy publishing. */

/** Absolute path to the WordPress directory. */
if ( ! defined( 'ABSPATH' ) ) {
        define( 'ABSPATH', __DIR__ . '/' );
}

/** Sets up WordPress vars and included files. */
require_once ABSPATH . 'wp-settings.php';
```

Revisamos el archivo .htaccess

```
debian@debian:~$ sudo ls -l /var/www/html/.htaccess
-rwxrwxrwx 1 www-data www-data 523 Sep 30  2024 /var/www/html/.htaccess
```

```
debian@debian:~$ sudo cat /var/www/html/.htaccess

# BEGIN WordPress
# The directives (lines) between "BEGIN WordPress" and "END WordPress" are
# dynamically generated, and should only be modified via WordPress filters.
# Any changes to the directives between these markers will be overwritten.
<IfModule mod_rewrite.c>
RewriteEngine On
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
RewriteBase /
RewriteRule ^index\.php$ - [L]
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule . /index.php [L]
</IfModule>
```

```
debian@debian:~$ sudo cat /var/www/html/wp-content/plugins/akismet/.htaccess
# Only allow direct access to specific Web-available files.

# Apache 2.2
<IfModule !mod_authz_core.c>
        Order Deny,Allow
        Deny from all
</IfModule>

# Apache 2.4
<IfModule mod_authz_core.c>
        Require all denied
</IfModule>

# Akismet CSS and JS
<FilesMatch "^(form\.js|akismet(-frontend|-admin)?\.js|akismet(-admin)?(-rtl)?\.css|inter\.css)$">
        <IfModule !mod_authz_core.c>
                Allow from all
        </IfModule>

        <IfModule mod_authz_core.c>
                Require all granted
        </IfModule>
</FilesMatch>

# Akismet images
<FilesMatch "^(logo-(a|full)-2x\.png|akismet-refresh-logo\.svg|akismet-refresh-logo@2x\.png|arrow-left\.svg|icon-e
xternal\.svg)$">
        <IfModule !mod_authz_core.c>
                Allow from all
        </IfModule>

        <IfModule mod_authz_core.c>
                Require all granted
        </IfModule>
</FilesMatch>
```

Desde Kali, validamos los recursos que son accesibles desde otra máquina.

```
┌──(kali㉿kali)-[~]
└─$ curl -I http://192.168.1.33/wp-login.php
HTTP/1.1 200 OK
Date: Tue, 17 Mar 2026 12:18:49 GMT
Server: Apache/2.4.62 (Debian)
Expires: Wed, 11 Jan 1984 05:00:00 GMT
Cache-Control: no-cache, must-revalidate, max-age=0, no-store, private
Set-Cookie: wordpress_test_cookie=WP%20Cookie%20check; path=/; HttpOnly
X-Frame-Options: SAMEORIGIN
Content-Security-Policy: frame-ancestors 'self';
Referrer-Policy: strict-origin-when-cross-origin
Content-Type: text/html; charset=UTF-8

┌──(kali㉿kali)-[~]
└─$ curl -I http://192.168.1.33/wp-json
HTTP/1.1 200 OK
Date: Tue, 17 Mar 2026 12:19:03 GMT
Server: Apache/2.4.62 (Debian)
X-Robots-Tag: noindex
Link: <http://localhost/index.php/wp-json/>; rel="https://api.w.org/"
X-Content-Type-Options: nosniff
Access-Control-Expose-Headers: X-WP-Total, X-WP-TotalPages, Link
Access-Control-Allow-Headers: Authorization, X-WP-Nonce, Content-Disposition, Content-MD5, Content-Type
Allow: GET
Content-Type: application/json; charset=UTF-8

┌──(kali㉿kali)-[~]
└─$ curl -I http://192.168.1.33/readme.html
HTTP/1.1 200 OK
Date: Tue, 17 Mar 2026 12:19:12 GMT
Server: Apache/2.4.62 (Debian)
Last-Modified: Fri, 13 Mar 2026 19:42:18 GMT
ETag: "1d01-64ced11260798"
Accept-Ranges: bytes
Content-Length: 7425
Vary: Accept-Encoding
Content-Type: text/html
```

Verificamos con Nmap HTTP

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -Pn -p 80 --script http-title,http-enum,http-security-headers 192.168.1.33 -oN ~/Desktop/pentest_resultados/60_nmap_http_pre.txt
sudo: unable to resolve host kali: Nombre o servicio desconocido
[sudo] contraseña para kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-17 13:20 +0100
Nmap scan report for 192.168.1.33
Host is up (0.00052s latency).

Bug in http-security-headers: no string output.
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.62 ((Debian))
|_http-title: Apache2 Debian Default Page: It works
|_http-server-header: Apache/2.4.62 (Debian)
| http-enum:
|   /wp-login.php: Possible admin folder
|   /wp-json: Possible admin folder
|   /robots.txt: Robots file
|   /readme.html: Wordpress version: 2
|   /wp-includes/images/rss.png: Wordpress version 2.2 found.
|   /wp-includes/js/jquery/suggest.js: Wordpress version 2.5 found.
|   /wp-includes/images/blank.gif: Wordpress version 2.6 found.
|   /wp-includes/js/comment-reply.js: Wordpress version 2.7 found.
|   /wp-login.php: Wordpress login page.
|   /wp-admin/upgrade.php: Wordpress login page.
|   /readme.html: Interesting, a readme.
|_  /0/: Potentially interesting folder
MAC Address: 08:00:27:8C:8C:A0 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 49.70 seconds
```

NIKTO

```
┌──(kali㉿kali)-[/]
└─$ perl /opt/nikto/program/nikto.pl -h http://192.168.1.33 -o ~/Desktop/pentest_resultados/61_nikto_pre.txt -Format txt
- Nikto v2.8.0
---------------------------------------------------------------------------
+ Target IP:          192.168.1.33
+ Target Hostname:    192.168.1.33
+ Target Port:        80
+ Platform:           Linux/Unix
+ Start Time:         2026-03-17 13:44:18 (GMT1)
---------------------------------------------------------------------------
+ Server: Apache/2.4.62 (Debian)
+ [000427] /cgi.cgi/: Link header(s) found with value(s): <http://localhost/index.php/wp-json/>; rel="https://api.w.org/". See: https://developer.mozilla.org/
en-US/docs/Web/HTTP/Headers/Link
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [999100] /wp-admin/: Uncommon header(s) 'x-redirect-by' found, with contents: WordPress.
+ [999996] /robots.txt: contains 2 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/Robots.txt
+ [000058] Apache/2.4.62 appears to be outdated (current is at least 2.4.66).
+ [999904] /: Server may leak inodes via ETags, header found with file /, inode: 29cd, size: 623573d915b52, mtime: gzip. See: https://cve.mitre.org/cgi-bin/cv
ename.cgi?name=CVE-2003-1418
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Se
curity
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [999990] OPTIONS: Allowed HTTP Methods: OPTIONS, HEAD, GET, POST .

+ [006184] /wp-links-opml.php: This WordPress script reveals the installed version.
+ [006186] /license.txt: License file found may identify site software.
+ [750500] /wp-content/uploads/: Directory indexing found.
+ [005622] /wp-content/uploads/: WordPress uploads directory is browsable. This may reveal sensitive information.
+ [006668] /wp-login.php: WordPress login found.
+ [999979] http://100.100.100.200/latest/meta-data/: IP address found in the 'location' header. The IP is "100.100.100.200". See: https://portswigger.net/kb/i
ssues/00600300_private-ip-addresses-disclosed
+ [007242] /: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: http
s://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
+ [007252] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MI
ME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 8170 requests: 16 errors and 19 items reported on the remote host
+ End Time:           2026-03-17 13:54:11 (GMT1) (593 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Gobuster

```
┌──(kali㉿kali)-[/]
└─$ gobuster dir -u http://192.168.1.33 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,bak -o ~/Desktop/pentest_resultados/62_gobuster_pre.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.1.33
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php,txt,html,bak
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 277]
.hta.php             (Status: 403) [Size: 277]
.hta.txt             (Status: 403) [Size: 277]
.hta.html            (Status: 403) [Size: 277]
.hta.bak             (Status: 403) [Size: 277]
.htaccess            (Status: 403) [Size: 277]
.htaccess.php        (Status: 403) [Size: 277]
.htaccess.html       (Status: 403) [Size: 277]
.htaccess.txt        (Status: 403) [Size: 277]
.htpasswd            (Status: 403) [Size: 277]
.htpasswd.bak        (Status: 403) [Size: 277]
.htpasswd.txt        (Status: 403) [Size: 277]
.htpasswd.html       (Status: 403) [Size: 277]
.htpasswd.php        (Status: 403) [Size: 277]
.htaccess.bak        (Status: 403) [Size: 277]
0                    (Status: 301) [Size: 0] [--> http://192.168.1.33/0/]
admin                (Status: 301) [Size: 0] [--> http://localhost/wp-admin/]
dashboard            (Status: 302) [Size: 0] [--> http://localhost/wp-admin/]
favicon.ico          (Status: 302) [Size: 0] [--> http://localhost/wp-includes/images/w-logo-blue-white-bg.png]
index.html           (Status: 200) [Size: 10701]
index.php            (Status: 301) [Size: 0] [--> http://192.168.1.33/]
index.html           (Status: 200) [Size: 10701]
index.php            (Status: 301) [Size: 0] [--> http://192.168.1.33/]
license.txt          (Status: 200) [Size: 19903]
login                (Status: 302) [Size: 0] [--> http://localhost/wp-login.php]
login.php            (Status: 302) [Size: 0] [--> http://localhost/wp-login.php]
readme.html          (Status: 200) [Size: 7425]
robots.txt           (Status: 200) [Size: 109]
robots.txt           (Status: 200) [Size: 109]
server-status        (Status: 403) [Size: 277]
sitemap.xml          (Status: 200) [Size: 502]
sitemap.xml.php      (Status: 200) [Size: 502]
```

```
sitemap.xml.txt      (Status: 200) [Size: 502]
sitemap.xml.html     (Status: 200) [Size: 502]
sitemap.xml.bak      (Status: 200) [Size: 502]
wp-admin             (Status: 301) [Size: 315] [--> http://192.168.1.15/wp-admin/]
wp-app.php           (Status: 403) [Size: 0]
wp-atom.php          (Status: 301) [Size: 0] [--> http://localhost/index.php/feed/atom/]
wp-blog-header.php   (Status: 200) [Size: 0]
wp-commentsrss2.php  (Status: 301) [Size: 0] [--> http://localhost/index.php/comments/feed/]
wp-config.php        (Status: 200) [Size: 0]
wp-content           (Status: 301) [Size: 317] [--> http://192.168.1.15/wp-content/]
wp-cron.php          (Status: 200) [Size: 0]
wp-feed.php          (Status: 301) [Size: 0] [--> http://localhost/index.php/feed/]
wp-includes          (Status: 301) [Size: 318] [--> http://192.168.1.15/wp-includes/]
wp-links-opml.php    (Status: 200) [Size: 226]
wp-load.php          (Status: 200) [Size: 0]
wp-login.php         (Status: 200) [Size: 4582]
wp-mail.php          (Status: 403) [Size: 2520]
wp-rdf.php           (Status: 301) [Size: 0] [--> http://localhost/index.php/feed/rdf/]
wp-rss.php           (Status: 301) [Size: 0] [--> http://localhost/index.php/feed/]
wp-register.php      (Status: 301) [Size: 0] [--> http://localhost/wp-login.php?action=register]
wp-rss2.php          (Status: 301) [Size: 0] [--> http://localhost/index.php/feed/]
wp-settings.php      (Status: 500) [Size: 0]
wp-signup.php        (Status: 302) [Size: 0] [--> http://localhost/wp-login.php?action=register]
wp-trackback.php     (Status: 200) [Size: 135]
xmlrpc.php           (Status: 405) [Size: 42]
xmlrpc.php           (Status: 405) [Size: 42]
Progress: 23065 / 23065 (100.00%)
=====================================================
Finished
=====================================================
```

Correguimos los problemas en Debian

Hacemos un backup

```
debian@debian:~$ sudo cp /var/www/html/wp-config.php /var/www/html/wp-config.php.bak.$(date +%F_%H%M&S)
bash: S: command not found
[sudo] password for debian:
Sorry, try again.
[sudo] password for debian:
```

Generamos una contraseña segura para usar

```
debian@debian:~$ openssl rand -base64 24
cMFXDMa7uIZGFBseaqP+afeGhkTfy2P4
```

Asignamos esa contraseña

```
debian@debian:~$ sudo mysql -e "ALTER USER 'wordpressuser'@'localhost' IDENTIFIED BY 'cMFXDMa7uIZGFBseaqP+afeGhkTfy2P4'; FLUSH PRIVIL
EGES;"
```

Modificamos la contraseña en el archivo wp-config.php

```
/** Database password */
define( 'DB_PASSWORD', 'Serv_TRAB:Fnal_MAAC' );
```

M odificamos las Salts

```
 */
define( 'AUTH_KEY',         '25 Color de tu pelo 98' );
define( 'SECURE_AUTH_KEY',  '65 Cada uno pone lo que quiere 25' );
define( 'LOGGED_IN_KEY',    '789 adivina adivinador, quien gana hoy' );
define( 'NONCE_KEY',        'la verdad 99 es que no se que poner90' );
define( 'AUTH_SALT',        'la realidad es que 89 es 22 y 4' );
define( 'SECURE_AUTH_SALT', 'Kiko, jijo, lito, todito00' );
define( 'LOGGED_IN_SALT',   '558799Al la mirada de todos' );
define( 'NONCE_SALT',       'nadie salva- 75 la 38 i npo' );
```

Modificamos permisos

```
debian@debian:~$ sudo chown root:www-data /var/www/html/wp-config.php
debian@debian:~$ sudo chmod 640 /var/www/html/wp-config.php
debian@debian:~$ sudo chmod 644 /var/www/html/.htaccess
debian@debian:~$ sudo chmod 644 /var/www/html/wp-content/plugins/akismet/.htaccess
debian@debian:~$ sudo find /var/www/html -type d -exec chmod 755 {} \;
debian@debian:~$ sudo find /var/www/html -type d -exec chmod 644 {} \;
debian@debian:~$ sudo chmod 640 /var/www/html/wp-config.php
```

Verificamos los permisos

```
debian@debian:~$ sudo ls -l /var/www/html/.htaccess /var/www/html/wp-content/plugins/akismet/.htaccess
-rw-r--r-- 1 www-data www-data 523 Sep 30  2024 /var/www/html/.htaccess
-rw-r--r-- 1 www-data www-data 772 Nov 20  2023 /var/www/html/wp-content/plugins/akismet/.htaccess
debian@debian:~$ sudo ls -l /var/www/html/wp-config.php
-rw-r----- 1 root www-data 3055 Mar 17 09:27 /var/www/html/wp-config.php
```

Verificamos que apache este activo

```
debian@debian:~$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-03-17 04:39:25 EDT; 4h 59min ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 709 (apache2)
      Tasks: 11 (limit: 9427)
     Memory: 153.4M
        CPU: 13min 55.698s
     CGroup: /system.slice/apache2.service
             ├─  709 /usr/sbin/apache2 -k start
             ├─  769 /usr/sbin/apache2 -k start
             ├─  771 /usr/sbin/apache2 -k start
             ├─  772 /usr/sbin/apache2 -k start
             ├─ 8594 /usr/sbin/apache2 -k start
             ├─ 8595 /usr/sbin/apache2 -k start
             ├─14540 /usr/sbin/apache2 -k start
             ├─14543 /usr/sbin/apache2 -k start
             ├─14544 /usr/sbin/apache2 -k start
             ├─14545 /usr/sbin/apache2 -k start
             └─14547 /usr/sbin/apache2 -k start

Mar 17 04:39:25 debian systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 17 04:39:25 debian systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Comprobamos que el servidor responda

```
debian@debian:~$ curl -I http://127.0.0.1
HTTP/1.1 403 Forbidden
Date: Tue, 17 Mar 2026 13:39:49 GMT
Server: Apache/2.4.62 (Debian)
Content-Type: text/html; charset=iso-8859-1

debian@debian:~$ curl -I http://192.168.1.33
HTTP/1.1 403 Forbidden
Date: Tue, 17 Mar 2026 13:40:34 GMT
Server: Apache/2.4.62 (Debian)
Content-Type: text/html; charset=iso-8859-1
```

Vamos a Ocular mejor la información con un bloque de Hardening

```
debian@debian:~$ sudo tee /etc/apache2/conf-available/hardening.conf > /dev/null <<'EOF'
ServerTokens Prod
ServerSignature Off
TraceEnable Off
Header Always unset X-Powered-By
Header Always set X-Content-Type-Options "nosniff"
Header Always set Referrer-Policy "strict-origin-when-cross-origin"
Header Always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
Header Always set Content-Security-Policy "default-src 'self'; frame-ancestors 'self'; form-action 'self'"

Header Always set X-Frame-Options "SAMEORIGIN"
EOF
```

También activamos módulos

```
debian@debian:~$ sudo a2enmod headers
Enabling module headers.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

```
debian@debian:~$ sudo a2enconf hardening
Enabling conf hardening.
To activate the new configuration, you need to run:
  systemctl reload apache2
```

Reiniciamos y verificamos el estatus de apache

```
debian@debian:~$ sudo systemctl restart apache2
debian@debian:~$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-03-17 10:32:57 EDT; 4s ago
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 21602 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Main PID: 21606 (apache2)
      Tasks: 6 (limit: 9427)
     Memory: 16.2M
        CPU: 51ms
     CGroup: /system.slice/apache2.service
             ├─21606 /usr/sbin/apache2 -k start
             ├─21607 /usr/sbin/apache2 -k start
             ├─21608 /usr/sbin/apache2 -k start
             ├─21609 /usr/sbin/apache2 -k start
             ├─21610 /usr/sbin/apache2 -k start
             └─21611 /usr/sbin/apache2 -k start

Mar 17 10:32:57 debian systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 17 10:32:57 debian systemd[1]: Started apache2.service - The Apache HTTP Server.
```

Verificamos en Kali que el servidor responda

```
┌──(kali㉿kali)-[/]
└─$ curl -I http://192.168.1.33/readme.html
HTTP/1.1 403 Forbidden
Date: Tue, 17 Mar 2026 13:41:24 GMT
Server: Apache/2.4.62 (Debian)
Content-Type: text/html; charset=iso-8859-1

┌──(kali㉿kali)-[/]
└─$ curl -I http://192.168.1.33/wp-login.php
HTTP/1.1 403 Forbidden
Date: Tue, 17 Mar 2026 13:41:39 GMT
Server: Apache/2.4.62 (Debian)
Content-Type: text/html; charset=iso-8859-1
```

Ejecutamos un Nikto otra vez para verificar (nikto detecta falso positivo en el X-Content-Type-Options header).

```
┌──(kali㉿kali)-[/]
└─$ perl /opt/nikto/program/nikto.pl -h http://192.168.1.33 -o ~/Desktop/pentest_resultados/61_nikto_post2.txt -Format txt
- Nikto v2.6.0
---------------------------------------------------------------------------
+ Target IP:          192.168.1.33
+ Target Hostname:    192.168.1.33
+ Target Port:        80
+ Platform:           Unknown
+ Start Time:         2026-03-17 19:22:52 (GMT1)
---------------------------------------------------------------------------
+ Server: Apache
+ No CGI Directories found (use '-C all' to force check all possible dirs), CGI tests skipped.
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Se
curity
+ [007352] /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MI
ME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 8237 requests: 16 errors and 2 items reported on the remote host
+ End Time:           2026-03-17 19:28:49 (GMT1) (357 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Vista de debian con los cambios que nikto detecta como falso positivo.

```
debian@debian:~$ curl -I http://192.168.1.33
HTTP/1.1 403 Forbidden
Date: Tue, 17 Mar 2026 18:22:05 GMT
Server: Apache
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Security-Policy: default-src 'self'; frame-ancestors 'self'; base-uri 'self'; form-action 'self'
X-Frame-Options: SAMEORIGIN
Content-Type: text/html; charset=iso-8859-1

debian@debian:~$ curl -I http://127.0.0.1
HTTP/1.1 403 Forbidden
Date: Tue, 17 Mar 2026 18:22:16 GMT
Server: Apache
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
Content-Security-Policy: default-src 'self'; frame-ancestors 'self'; base-uri 'self'; form-action 'self'
X-Frame-Options: SAMEORIGIN
Content-Type: text/html; charset=iso-8859-1
```

Ejecutamos un Gobuster otra vez para verificar

```
┌──(kali㉿kali)-[/]
└─$ gobuster dir -u http://192.168.1.33 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html,bak --exclude-length 339 -o ~/Desktop/pentest_resultados/02_go
buster_post2.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.1.33
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] Exclude Length:          339
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php,txt,html,bak
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
Progress: 23065 / 23065 (100.00%)
===============================================================
Finished
===============================================================
```

Eliminación del Usuario “dudoso”

```
debian@debian:~$ sudo mysql -e "SHOW GRANTS FOR `user`@`localhost`;"
+-----------------------------------------------------------------------------------------------------------------------------------+
| Grants for user@localhost                                                                                                         |
+-----------------------------------------------------------------------------------------------------------------------------------+
| GRANT ALL PRIVILEGES ON *.* TO `user`@`localhost` IDENTIFIED BY PASSWORD '*2470C0C06DEE42FD1618880990B5ADCA2EC9D1E19' WITH GRANT OPTION |
+-----------------------------------------------------------------------------------------------------------------------------------+
```

```
debian@debian:~$ sudo mysql -e "DROP USER 'user'@'localhost';"
```

Comprobación de la eliminación

```
debian@debian:~$ sudo mysql -e "SELECT User, Host FROM mysql.user ORDER BY USer, Host;"
+---------------+-----------+
| User          | Host      |
+---------------+-----------+
| mariadb.sys   | localhost |
| mysql         | localhost |
| root          | localhost |
| wordpressuser | localhost |
+---------------+-----------+
```

Instalación de rsyslogs

```
debian@debian:~$ sudo apt install -y rsyslog
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

```
debian@debian:~$ sudo systemctl status rsyslog --no-pager
● rsyslog.service - System Logging Service
     Loaded: loaded (/lib/systemd/system/rsyslog.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-03-23 14:20:29 EDT; 2min 23s ago
TriggeredBy: ● syslog.socket
       Docs: man:rsyslogd(8)
             man:rsyslog.conf(5)
             https://www.rsyslog.com/doc/
   Main PID: 23791 (rsyslogd)
      Tasks: 4 (limit: 9410)
     Memory: 5.8M
        CPU: 14ms
     CGroup: /system.slice/rsyslog.service
             └─23791 /usr/sbin/rsyslogd -n -iNONE

Mar 23 14:20:29 debian systemd[1]: Starting rsyslog.service - System Logging Service...
Mar 23 14:20:29 debian rsyslogd[23791]: imuxsock: Acquired UNIX socket '/run/systemd/journal/syslog' (fd 3) from systemd.  [v8.2302.0]
Mar 23 14:20:29 debian rsyslogd[23791]: [origin software="rsyslogd" swVersion="8.2302.0" x-pid="23791" x-info="https://www.rsyslog.com"] start
Mar 23 14:20:29 debian systemd[1]: Started rsyslog.service - System Logging Service.
```